The Brisket Dynasty Valuation Model (BDVM) v1.0

A projection-driven, format-configurable dynasty valuation framework for offense and IDP

Author: research + design document prepared for RiskItToGetTheBrisket.org **Date:** July 2026 (targeting the 2026 NFL season as "upcoming") **Status:** Design complete. Reference implementation included and executed. All weights are **starting priors requiring backtesting**, not proven values.

This is a single self-contained document. Parts 1–14 are the research and design. Appendices A–F contain the complete source of every companion file, embedded verbatim — copy each fenced block into the filename given in its heading and everything runs.

Appendix	File	What it is
A	`dynasty_engine.py`	Runnable stdlib-only reference implementation of every formula in Part 5 (702 lines, no dependencies)
B	`run_examples.py`	Driver that produces the Part 10 worked examples
C	`examples_output.txt`	Actual computed output of A + B
D	`schema.sql`	PostgreSQL schema for the ingestion + calculation pipeline
E	`league_config.example.json`	Your 12-team SF / TEP / PPR / IDP league encoded as config
F	`player_payload.example.json`	API request/response contract for the projection ingest

To split it back into files: save Appendix A's block as `dynasty_engine.py` and Appendix B's as `run_examples.py` in the same directory, then `python3 run_examples.py` reproduces Part 10 exactly. Nothing to install.

How to read this document. Claims sourced to outside research are linked inline. Anything labelled **[Recommendation]** or **[Hypothesis]** is my design judgment, not an established finding. Anything labelled **[Prior]** is a starting weight that must survive backtesting before you trust it.

Part 1 — Executive summary

1.1 The one-paragraph version

Dynasty value is the **discounted expected sum of future seasons of surplus starter production, weighted by the probability the player is still startable in each of those seasons**. Everything else — age, draft capital, contract, college production, scheme, coaching — is a *modifier on one of three things*: (a) the level of the production curve, (b) the shape/slope of that curve over time, or (c) the probability of surviving to collect it. Organising the model that way is what stops it from becoming an arbitrary weighted-average bonus pile.

1.2 The central formula

For player *i* under strategy profile *s*:

```
                  H
DV(i,s)  =  Σ    u_s(t) · d_s^t · S_i(t) · E[ max(0, μ_i(t) − R_p(t)) ] · G_i(t)
            t=0
          − λ_s · Ψ_i          (risk / certainty-equivalent adjustment)
```

Term	Meaning
`μ_i(t)`	Expected fantasy **points per game** in season *t*, from the blended projection propagated forward by the aging + ascension engine
`R_p(t)`	Dynamic **replacement level** (points per game) for the player's lineup group, derived from league size, starters, flex eligibility and waiver depth
`G_i(t)`	Expected games played
`E[max(0, μ−R)]`	Expected **starter surplus**, evaluated under the player's outcome distribution — not `μ−R`. This is the single most important design choice; see §1.4
`S_i(t)`	Probability the player is still fantasy-startable in season *t* (discrete-time survival, product of annual hazards)
`d_s`	Per-year discount factor for strategy *s*

`u_s(t)`	**Usability weight** — how much a point scored in season *t* is worth *to this specific roster* (a rebuilder does not fully value this year's points)
`λ_s · Ψ_i`	Certainty-equivalent adjustment; contenders are risk-averse (λ>0), rebuilders mildly risk-seeking (λ<0)

The model runs three times per player (contender / balanced / rebuilder) and emits three separate values, plus a market-blended value and a model-vs-market gap.

1.3 The nine modules

This is close to the modular structure you proposed. Three changes I recommend:

1. **Merge "Aging-Curve Engine" and "Career Longevity Engine" conceptually but keep them numerically separate.** They answer different questions — *how good is he if he plays* vs *does he play* — and conflating them is the #1 source of double-counting in public dynasty models (see §2.2).
2. **Add a Role-Ascension sub-module.** Without it, every young player is penalised for having a low current projection, which makes rookies and second-year breakout candidates systematically undervalued.
3. **Demote "Team-Strategy Adjustment Engine" from a post-hoc multiplier to a set of parameters (`d_s` , `u_s` , `λ_s`) inside the core sum.** Applying a contender/rebuilder multiplier *after* computing a single dynasty number loses the horizon structure that makes the distinction meaningful.

1.4 The one non-obvious design decision

Do not use `μ − R` . Use `E[max(0, μ − R)]` .

You are never forced to start a below-replacement player. That truncation means a player's contribution is a **call option on his own production**, struck at replacement level. Under `μ(t) ~ Normal(μ, σ)` :

```
E[max(0, μ−R)] = (μ−R)·Φ(z) + σ·φ(z)      where z = (μ−R)/σ
```

Consequences, all of which you explicitly asked for and none of which need a hand-tuned bonus:

- Two players projected for identical points but with different outcome ranges get **different values** — the wide one is worth more when near/below replacement, and essentially the same when far above it.
- Rookies and unproven young players get real value from **upside** rather than from a "youth multiplier."
- Volatile IDP archetypes (boundary CBs, sack-dependent EDGE) are handled by the same math as offense.
- Risk stops being a fudge factor and becomes a parameter of a distribution.

Separately, **attrition risk is multiplicative** (`S(t)`), and **manager risk preference** is a final certainty-equivalent shift (`λ_s`). Three different kinds of "risk," three different mechanisms. That separation is the core of my answer to "should risk reduce value or be shown separately": **outcome**

dispersion is priced in (it raises value at the bottom, is neutral at the top), attrition risk reduces value, and risk *tolerance* is a user-facing slider.

1.5 Version 1 recommendation in one line

Build the transparent parametric model above with position-specific curves and a dynamic replacement engine — not a gradient-boosted model. Full justification in Parts 7 and 14. The short reason: your target variable (multi-year discounted future value) has ~1 observation per player per year, IDP sample sizes at DT/CB are small, and you need every number to be explainable on a player card. A glass-box model that you can backtest and calibrate will beat an unexplainable GBM that you can't debug, and it is the only version you can ship in weeks rather than months.

Part 2 — Research findings

Everything in this part is sourced. Where the public research is thin or contradictory, I say so.

2.1 Aging curves are real, position-specific, and routinely mis-estimated

Established findings.

- **Running backs peak earliest and decline fastest.** 4for4's production-curve study places the average back's best season around **age 26**, with production roughly flat from entry through age 28, and finds the single most common breakout season for a back is his **rookie year** — the opposite of the receiver pattern (4for4, 2025). An Apex study using a 15 PPR/game threshold since 2000 puts average RB peak age at **25.46 vs 26.95 for WRs**, and notes only 2 qualifying RB seasons at age 32+ against 37 such WR seasons (Apex). FantasyPros found ~**40.6%** of age-21 RBs post a top-24 season — the highest rate on their chart — with a clear top-24 drop after the age-28 season (FantasyPros, 2021).
- **Wide receivers ramp slowly and hold longer.** PFF's league-year curves show receivers commonly needing two to four years to reach their ceiling, in contrast to backs (PFF).
- **Tight ends have the largest documented year-2 jump of any position.** PFF measured sophomore TEs moving from ~33% to ~94% of their career baseline average (PFF); ESPN's version puts the rookie→sophomore PPR/game increase at roughly **98.5%**, with the level then holding through about the seventh season and no significant decline until around age 30 (ESPN). FTN's over-30 study finds TEs have aged *better* over the last two decades while RBs and WRs over 30 fell off sharply (FTN).
- **Quarterbacks peak latest — but only some of them.** Classic work puts QB peak around **age 29** with a 26–30 prime (Football Perspective), and Advanced Football Analytics' delta-method analysis finds established 10+ year starters peaking near 29 with a shallow decline (AFA). The critical modern refinement: an Apex study of QB seasons since 2000 above 18 PPR/game finds an overall peak of **28.5**, splitting into **30.7 for pocket passers vs 26.1 for dual-threats**, with dual-threat QBs accounting for only ~4.1% of qualifying peak seasons at 33+ and QB rushing production dropping sharply after 29 (Apex).

The methodological warning that matters most. Naive aging curves are contaminated by survivor bias: if you average the production of everyone active at each age, you are averaging *survivors*, and if you build the curve from players' final seasons you get a decline that is partly a selection artifact. Harstad's analysis found that among the top 50 RBs and top 50 WRs by career fantasy value, exactly **50 of 100** declined in their final fantasy-relevant season — i.e., no better than a coin flip — which is not what a naive curve predicts (Footballguys). Advanced Football Analytics reaches the same conclusion from the other direction, showing that different sample-construction choices produce materially different QB curves and that the delta method (year-pair changes) is needed to remove selection effects (AFA).

[Recommendation] Build your curves with the **delta method on consecutive year-pairs, conditional on the player playing a meaningful snap threshold in both years**, then handle disappearance separately with a survival model. If you build a single curve on all players including zeros, you have silently merged decline-in-production with loss-of-job, and every subsequent risk adjustment you add will double-count.

2.2 What "already in the projection" means — the double-counting audit

This is the discipline the whole model depends on. A credible upcoming-season projection **already contains**: team quality, offensive scheme, quarterback quality, offensive-line quality, current depth chart, expected snap share, expected target/carry share, coaching change effects, and a games-played haircut for known injuries.

Therefore:

Variable	Re-add to *this season's* points?	Legitimate use
Team quality / scheme / QB play	**No**	Only as an input to *future*-season role stability and ascension
Snap / route / target share	**No** for level	Yes for **role security** (survival) and **σ** (uncertainty)
Coaching staff change	**No**	Yes as a **news event** with decay, and as scheme-stability risk

Injury designation now	**No**	Yes for **future** $G(t)$ and durability
Age	**Yes** — projections are single-season, they contain no multi-year decline	Trajectory engine
Draft capital	**Partially** — good projections use it	Restrict to survival + ascension, and **decay it by NFL season**
College production	**Mostly no** after year 1	Rookie/year-2 ascension only, with an explicit decay schedule
Contract	**No** for this season	Survival, and future role security

[Recommendation] Encode this as a hard rule in code: *any variable that plausibly enters the projection may only affect* $S(t)$, $\sigma(t)$, κ *(ascension), or* $G(t \geq 1)$ *— never* $\mu(0)$. That single rule prevents most of the overfitting risk you were worried about.

2.3 Which statistics are actually predictive (and therefore worth storing)

Offense — receiving. Volume beats efficiency for year-over-year stability. Sharp Football's stickiness work finds fantasy points per game the most stable receiver metric year to year, target-based counting stats close behind, yards per team pass attempt and team target share the stickiest rate stats, and per-target efficiency (YPT, YAC, TD rate) the least stable; games played is the *weakest* correlate year to year (Sharp Football). SumerSports reports year-to-year stability of roughly **0.51 for YPRR**, **0.67 for expected YPRR** (falling to **0.49** when the receiver changes teams), around **0.70 for target share**, and about **0.60 for QB EPA per attempt** (SumerSports YPRR, SumerSports sticky stats). PFF measured YPRR at a **0.43** correlation with next-season receiving fantasy points versus **0.46** for raw targets and **0.18** for yards per target (PFF).

Offense — tight end. Fantasy Life's 2026 TE study reports receiving TDs correlating **0.74** with same-season PPG but only **0.33** with next-season PPG (TD year-over-year stability ~**0.28**), while receiving yards per game carries a **0.65** year-over-year correlation; among high-route-share TEs, YPRR rises to about **0.65** predictive correlation (Fantasy Life).

Implication for the model. Touchdown-dependence is a *volatility* input, not a value input. A player whose projection is TD-heavy should carry a higher σ and a modest downward regression in $\mu(t \geq 1)$, not a haircut today.

IDP. The consensus of current IDP analytics is unusually clear and unusually consistent across outlets:

- **Snap share is the first-order variable.** Full-time defenders have far more reliable tackle floors than rotational specialists; three-down linebackers who stay on the field in passing situations protect both floor and coverage upside (Yahoo/Athlon/Lindy's syndicated IDP explainer).
- **Tackles are more stable than sacks.** PFF's older linebacker study found the top of the sack leaderboard turns over roughly twice as fast as the top of the tackle leaderboard (PFF). Modern practice has formalised this via **expected sacks** (built from pass-rush snaps plus rolling grade/win-rate/pressure inputs), described as the most stable pass-rush metric year to year and used explicitly as a sack-regression signal, and **tackles vs expected**, which adjusts raw tackle counts for snaps, role and scheme (Fantasy Life IDP, The IDP Show).
- **Alignment dominates for defensive backs.** Box and slot safeties get steady tackle chances; deep/centerfield safeties are big-play dependent and have weak floors; Cover-2 shells keep safeties near the line while Cover-3/Cover-4 push them deep (Fantasy Life).
- **Cornerback fantasy production is partly *inverse* to real-life quality.** Elite shutdown corners get avoided, which suppresses tackles, breakups and interceptions; frequently targeted corners accumulate opportunity. Pass breakups are more repeatable than interceptions (Yahoo IDP explainer, NFL.com). Rookie corners are frequently tested, giving them a temporary opportunity spike that fades as they earn respect.
- **Position designation is a first-class risk.** PFF's dynasty IDP rankings explicitly assume "true position" (DT / EDGE / LB / CB / S), treating 3-4 outside linebackers as edge defenders (PFF) — but many league platforms do not, and a player's eligibility can flip between seasons. That flip can move a player between a scarce group and a deep one.
- **Age.** Public IDP aging work is thinner than offensive work. What exists suggests a peak band of roughly **25–30** for top defensive-line scorers, with interior linemen holding value later and linebackers more exposed to scheme change (Fantasy Six Pack). DraftSharks states it fits position-specific aging curves and expected retirement rates from two decades of data for its IDP dynasty product, though the parameters are proprietary (DraftSharks).

[Hypothesis] IDP aging is better modelled as **role aging** than physical aging: a linebacker's fantasy collapse is usually a snap-share event (new coordinator, drafted competition), not a gradual decline. That argues for a *higher hazard rate and a flatter production curve* at LB/CB than at WR — which is how the priors in Part 6 are set.

2.4 Draft capital: strong early, decaying, and mostly about opportunity

- The Fantasy Footballers' study of 1,349 players from the 2000–2018 classes found first-round RBs averaging **13.06 PPR points per game** across their first three seasons versus **8.59** for day-two backs (~34% higher), and first-round TEs at about **7.49** PPG on ~4.51 opportunities per game (Fantasy Footballers).
- Draft capital also predicts *career survival*, not just production: first-rounders play nearly twice as many games as seventh-rounders, and first-round

retention rates stay near ~85% several years in (IronMan Performance analysis).

- Dynasty-side syntheses consistently reach the same conclusion — rounds 1–2 carry the highest hit rates, and capital continues to buy patience and opportunity after a poor rookie year (Dynasty Nerds).

[Recommendation] Model draft capital as affecting **survival and ascension only**, with an explicit exponential decay in NFL season: `w_dc(n) = exp(−0.45·(n−1))`. That leaves roughly 64% of the effect in year 2, 41% in year 3, 26% in year 4, ~10% by year 6 — matching the intuition that first-round capital excuses one bad season but not four. After a player has 2+ years of established production, **recent team investment (extension, guaranteed money, snap share) should replace draft position** as the role-security signal.

2.5 College production: useful, and expires

Breakout age (first season at ≥20% college dominator rating) and dominator rating came out of RotoViz work by Siegele and DuPont and are now standard prospect inputs (PlayerProfiler, PFF). Apex's profile of rookie-WR breakouts found an average college breakout age of **19.77**, average peak dominator of **33.6%**, average NFL draft slot of **40.4**, and **83.9%** coming from rounds 1–2 (Apex).

[Recommendation] Weight college inputs `w_col(n) = exp(−1.1·(n−1))`: full weight as a rookie, ~33% in year 2, ~11% in year 3, effectively zero from year 4. Once a player has a meaningful NFL sample (≥ 250 routes, ≥ 150 carries, ≥ 400 defensive snaps), NFL evidence should dominate. Continuing to apply a college-production bonus to a fourth-year pro is one of the most common errors in public dynasty models.

2.6 Injuries: history predicts recurrence, not general fragility

- Systematic-review evidence in field sports is unambiguous that **previous injury is the strongest single predictor of future injury of the same type** — hamstring recurrence odds ratios in the 4x–11x range in elite football populations (systematic review, PMC; prospective elite-football study).
- Player-level susceptibility exists beyond load and prior injury: one prospective study found "injury-prone" players carried a hazard ratio above 5 even after adjusting for training load and injury history (PMC).
- On the fantasy side, Draft Sharks reports that in their games-missed model, **time elapsed since the last soft-tissue injury** ranked among the most important inputs while cumulative injury *counts* did not (Draft Sharks). The counter-position — that "injury prone" is largely a narrative and that risk is injury-specific rather than general — is argued at Fantasy Points.

[Recommendation] Do **not** use a raw count of career injuries. Use: (1) recency-weighted games missed over the last three seasons, (2) a same-type recurrence flag, (3) time since last soft-tissue event. Feed these into `G(t)` and `durability`, not into `μ`.

2.7 Replacement level, not raw points, is the unit of value

Value-Based Drafting — normalising every player against a positional baseline so a QB can be compared with a RB — traces to Joe Bryant at Footballguys around 2001 and remains the standard framework (FantasyPros VBD primer). The three common baselines are VORP (best freely available waiver player), VOLS (worst starter), and VONA (next available at your next pick). FantasyPros' worked 2025 example — an elite WR at ~169 half-PPR points of VORP against the top QB at ~127 despite the QB's much larger raw total — is the cleanest illustration of why raw projections cannot be compared across positions (FantasyPros, 2026 update).

This is the direct answer to your linebacker-versus-safety requirement. In a tackle-heavy IDP format, LB is both the highest-scoring and the *deepest* defensive group. A high replacement level compresses LB surplus; a lower DB replacement level expands safety surplus. The engine reproduces this: with the demo pool, LB replacement sits at **7.32 FPG** and DB at **6.17 FPG**, so a 12.5-FPG linebacker (surplus 5.18) is worth **less** than an 11.5-FPG box safety (surplus 5.33) despite scoring a point more per game.

[Recommendation] Use **VORP with a waiver-depth buffer** as the primary baseline, not VOLS. VOLS is too sensitive to the exact bottom starter, and dynasty value should reflect what you can actually replace a player with. Report VOLS as a secondary diagnostic.

2.8 Projection sources: blend them, weight by measured accuracy

Fantasy Football Analytics has run the longest-running public accuracy comparison of projection sources, using mean absolute error and MASE, and finds that (a) source accuracy varies materially **by position and by tier within position**, (b) no single source wins everywhere, and (c) an **accuracy-weighted average trained on prior seasons** is consistently at or near the top of the leaderboard — in their DFS study, the weighted blend led WR accuracy over eleven seasons at 4.94 MAE, marginally ahead of the best individual sources (FFA seasonal, FFA DFS, FFA positional bias).

[Recommendation] Weighted trimmed mean, weights refreshed annually from out-of-sample MAE **per position** (see §5.1). The gain over a simple average is real but small; the gain over a single source is large.

2.9 The dynasty market: three different things called "value"

- **KeepTradeCut** is pure crowdsourced preference — users repeatedly rank three assets, generating continuously updated 1QB and Superflex values, with TE-premium tiers (KTC).

- **FantasyCalc** derives values from **completed trades in real leagues** — a revealed-preference market rather than a stated-preference one.
- **DynastyProcess** builds its calculator on FantasyPros dynasty consensus expert rankings (DynastyProcess).

These are structurally different signals: stated preference, revealed preference, and expert consensus. They disagree in predictable ways — crowdsourced values overweight recent narrative and youth; trade-derived values reflect what actually clears; expert rankings are stickier and slower.

[Recommendation] Ingest at least one of each type and treat **their dispersion as your liquidity/uncertainty measure**, which is far more useful than any one of them as a point estimate. You already aggregate ten sources — keep that, but tag each source with its *type* so the dispersion measure is meaningful.

2.10 Rookie picks are distributions, and the distribution is steep

- NBC's historical work on rookie-pick outcomes found roughly **18%** of drafted rookies (91 of 504) ever recorded even one season as a top-12 QB/TE or top-24 RB/WR, with only ~6% doing it as rookies (NBC Sports).
- More recent tier-based work (Garret Price / Dynasty Nerds, 2018–2023 classes) reports very high hit rates at 1.01 and roughly three-quarters for top-four picks, then a largely undifferentiated remainder of round 1, a sharp collapse in round 2, and round 3 as effectively lottery tickets (Dynasty Nerds, methodology summary).

Note the tension: the two studies use different eras and different "hit" definitions, and the recent-era numbers are inflated by incomplete careers. **[Recommendation]** Build your own pick-outcome distributions from your own historical value database rather than importing either number, and treat published hit rates as sanity checks, not parameters.

2.11 Career length: the base rate is brutal

League-wide average NFL career length is commonly cited around **3.3 years**, with running backs shortest (~2.57), quarterbacks longest among non-specialists (~4.44), defensive linemen ~3.24 and linebackers ~2.97 (Statistico summary; Wikipedia summary of the same study). These averages include camp bodies and are *not* the right base rate for an established fantasy starter — but they are the right reminder that **the default outcome for a fantasy-relevant defensive player three years out is "not fantasy relevant."** Your survival module should be calibrated on *your* population (players who were startable in year 0), not on the league-wide average.

Part 3 — Data dictionary

Legend — **Req**: R = required, O = optional, D = desirable. **DC risk** = double-counting risk with the projection. **Leak** = data-leakage risk in backtesting.

3.1 Identity and biography

Variable	Definition	Type	Positions	Req	Purpose	Source	Update	DC risk	Leak
`player_id`	Canonical ID (GSIS preferred)	str	all	R	join key	nflverse `load_ff_playerids`	season	–	–
`sleeper_id`, `ktc_id`, `fc_id`	External IDs	str	all	R	market joins	crosswalk table	weekly	–	–
`birth_date`	DOB	date	all	R	exact age at season start	nflverse `load_players`	season	–	–
`age_season`	Age on Sept 1 of the season	float	all	R	aging curve input	derived	season	Low	–
`pos_true`	DT/EDGE/LB/CB/S/QB/RB/WR/TE	enum	all	R	curve + scarcity	PFF/SIS/manual review	season	–	–
`pos_platform`	Position as your league platform lists it	enum	all	R	eligibility + replacement group	Sleeper API	weekly	–	–
`archetype`	pocket/dual; box/deep; slot/boundary; 3-down/rotational	enum	QB,S,CB,LB,EDGE	D	curve selection	derived from snap alignment	season	Low	–
`nfl_season`	1 = rookie year	int	all	R	decay schedules	derived	season	–	–

`entry_year`, `draft_round`, `draft_pick`	Draft capital	int	all	R	ascension + survival	nflverse `load_draft_picks`	season	**Med**	–

3.2 Projection inputs (the swappable layer)

Variable	Definition	Type	Positions	Req	Purpose	Source	Update	DC risk	Leak
`proj_source`	Source name	str	all	R	weighting	your ingest	weekly	–	–
`proj_stat_line`	Raw projected stats (JSON)	json	all	D	internal scoring	source	weekly	–	–
`proj_fpts`	Projected season fantasy points	float	all	R*	fallback if no stat line	source	weekly	–	–
`proj_games`	Projected games played	float	all	R	`G(0)`	source	weekly	–	–
`proj_fpg`	Points per game	float	all	R	`μ(0)`	derived	weekly	–	–
`proj_snap_share`, `proj_route_pct`, `proj_tgt_share`, `proj_carry_share`	Projected opportunity	float	pos-specific	D	role security, σ	source	weekly	**High for μ**	–
`proj_high`, `proj_low`	Source ceiling/floor	float	all	O	σ calibration	source	weekly	–	–
`source_weight`	Accuracy weight by position	float	all	R	blending	derived from history	annual	–	**Yes** — must be trained only on prior seasons

`R*` = required if no raw stat line is provided.

3.3 Historical performance (for role, risk and backtest targets)

Variable	Definition	Type	Positions	Req	Purpose	Source	Update	DC risk	Leak
`fpg_prev1..3`	PPG in each of last 3 seasons (league scoring)	float	all	R	trend, σ, production z	nflverse `load_player_stats`	weekly	Med	–
`games_prev1..3`	Games played	int	all	R	durability	nflverse	weekly	–	–
`snap_pct_prev1..3`	Snap share	float	all	R	role security	nflverse `load_snap_counts` (PFR)	weekly	Med	–
`route_pct`, `tprr`, `yprr`, `tgt_share`, `ay_share`, `first_down_share`	Receiving role/efficiency	float	WR,TE,RB	D	role security, σ, regression	PFF/FTN/SIS or derived from PBP	weekly	Med	–
`carry_share`, `goalline_share`, `career_touches`	Rushing role + mileage	float	RB,QB	R	mileage, role	nflverse PBP	weekly	Med	–
`td_rate_vs_expected`	TD dependence	float	off	D	σ, mean-reversion	derived (`load_ff_opportunity`)	weekly	Med	–

`exp_fpts_prev`	Expected fantasy points (opportunity-based)	float	off	D	separates role from luck	nflverse `load_ff_opportunity`	weekly	Med	–
`pass_rush_snaps`, `pressure_rate`, `expected_sacks`, `qb_hits`, `hurries`	Pass-rush role/efficiency	float	EDGE,DT,LB	D	IDP μ regression + σ	PFF/SIS (paid) or PBP proxies	weekly	Med	–
`tackles_solo/ast`, `tackle_rate`, `tackles_vs_expected`	Tackle role/efficiency	float	LB,S,CB,DT,EDGE	R	IDP floor	nflverse defense stats + snaps	weekly	Med	–
`box_snap_pct`, `slot_snap_pct`, `deep_snap_pct`, `coverage_snaps`	DB alignment	float	S,CB,LB	D	archetype + floor	PFF/SIS	weekly	Med	–
`targets_faced`, `pass_breakups`, `int_rate`	CB opportunity	float	CB,S	D	CB-specific model	PFF/SIS	weekly	Med	–
`green_dot`	Defensive play-caller	bool	LB	D	snap-share lock	manual / beat reports	season	Low	–

3.4 Context, contract and risk

Variable	Definition	Type	Positions	Req	Purpose	Source	Update	DC risk	Leak
`contract_years_left`	Years remaining	int	all	R	survival	nflverse `load_contracts` (OverTheCap)	weekly	Low	–
`guaranteed_remaining`	Remaining guarantees	float	all	D	role security	OverTheCap	weekly	Low	–
`dead_cap_if_cut`	Cut cost	float	all	D	cap-casualty flag	OverTheCap	weekly	Low	–
`fifth_year_option`, `franchise_tag`, `rfa_status`	Contract state	bool/enum	all	D	survival	OverTheCap	weekly	Low	–
`depth_chart_rank`	Platform depth chart position	int	all	D	role security	nflverse `load_depth_charts`	weekly	**High**	–
`competition_added`	Draft pick or FA signing at same position	float	all	D	role risk event	derived from draft + transactions	event	Med	–
`coord_change`	New OC/DC this offseason	bool	all	D	scheme-stability risk	manual/scrape	season	Med	–
`games_missed_3y_wtd`	Recency-weighted missed games	float	all	R	durability	nflverse `load_injuries` + rosters	weekly	Med	–
`soft_tissue_recency`	Seasons since last soft-tissue injury	float	all	D	durability	injury log	weekly	Med	–
`combine_*` / RAS	Athletic testing	float	all	O	rookie ascension only	nflverse `load_combine`	season	Low	–
`college_dominator`, `breakout_age`, `col_ypta`,	College	float	off skill	D	rookie/year-	CFB data / prospect	season	Low	–

`early_declare`	production				2 ascension	DBs			
`col_pressure_rate`, `col_tackle_rate`, `col_coverage_grade`	College IDP production	float	IDP	O	rookie IDP ascension	CFB / PFF college	season	Low	–

3.5 Market

Variable	Definition	Type	Req	Purpose	Source	Update	Leak
`mkt_value_{source}`	Value on 0–10000 scale, per source	float	R	market layer	your existing 10-source scraper	daily	**Yes** — snapshot with `as_of_date`
`mkt_type`	crowd / trade-derived / expert	enum	R	dispersion typing	config	–	–
`mkt_dispersion`	Cross-source SD ÷ mean	float	R	liquidity, blend weight	derived	daily	–
`mkt_momentum_30d`	30-day change in consensus	float	D	buy/sell timing	derived	daily	–
`startup_adp`, `rookie_adp`	ADP by format	float	D	secondary market signal	ADP feeds	weekly	–

3.6 Variables I recommend you **exclude** from v1

Variable	Why excluded
Raw "team quality" / win totals	Already in projections; adding it double-counts, and Vegas-driven team quality is close to noise for individual dynasty value
Strength of schedule beyond the current season	Effectively unpredictable more than one season out; keep it in ROS only
Coaching *quality* ratings	No reliable public quantification; use scheme *stability* (binary change flag) instead
Raw career injury count	Superseded by recency-weighted missed games and same-type recurrence
Combine athleticism for players with 2+ NFL seasons	Dominated by NFL snap data; retaining it invites overfitting
Social/news sentiment scores	Unstable, gameable, and largely already in the market layer
Any in-season stat for a *preseason* backtest feature	Look-ahead leakage

---

Part 4 — Position-specific model design

Every position uses the same core formula. What differs is: which curve, which mileage unit, which risk drivers, which regression targets, and which σ multipliers.

4.1 Quarterback

Curve. Two curves, selected by archetype. `QB_pocket` peaks at 30 with a shallow decline; `QB_dual` peaks at 26 with a steep one, because the rushing component collapses first (Apex).

Archetype classification. `dual` if projected rushing points ≥ 20% of projected total, or prior-season designed-run rate above the QB median. Store as a *continuous* blend weight `w_dual ∈ [0,1]` and interpolate between the two curves — a hard flag mis-handles the many QBs in between.

Key risk drivers. Starting-job security dominates everything else; a benched QB goes from ~22 FPG to 0. Model `role_security` from: incumbent status, contract guarantees, draft capital of the backup, and whether the team drafted a QB in rounds 1–2 in the last two years.

Superflex. Handled entirely by the replacement engine — no QB multiplier anywhere in the model. In the demo config, Superflex raises QB replacement to 12.40 FPG, which correctly compresses QB surplus while the QB's superior aging curve and low hazard rate carry the long-horizon

value. **Note the expected disagreement:** pure VBD says elite QBs are *less* valuable than the Superflex market prices them. Do not suppress this — it is one of the model's most interesting outputs, and it is a real, longstanding VBD finding (FantasyPros 2026).

σ drivers. Rushing-TD dependence, offensive-line change, receiver-room turnover.

4.2 Running back

Curve. Peak 25, steep decline. **Mileage matters more here than anywhere else:** effective age = chronological age + `min(2.5, (career_touches − 750)/500)` . In the worked example, a 29-year-old with 1,720 career touches is evaluated at effective age **30.9**, and his value beyond year 1 rounds to zero.

Key drivers. Receiving role is the single best insulation — a back with real route participation retains value through committee changes and negative game scripts. Goal-line share drives TD variance (raise σ, do not raise μ beyond the projection).

Risk. Highest base hazard of any position at every age past 25. Contract status matters more than at other positions because RB second contracts are rare and cheap.

[Recommendation] Add a hard-coded **sell-window flag**: any RB whose effective age will exceed 27.5 within two seasons and whose market value is above the model's balanced value gets flagged. Public research on when to move off backs is consistent enough to justify a rule (PFF, Apex).

4.3 Wide receiver

Curve. Peak 27, slow ramp (young receivers are below their own peak), moderate decline. This is why the ascension term matters most at WR: a 23-year-old projected for 14 FPG is *not* a 14-FPG player — the curve alone lifts him ~23% by 26, before any role growth.

Key drivers. Target share and targets-per-route-run for role security; air-yard share for σ; team-change flag reduces the reliability of prior efficiency (expected YPRR stability falls from ~0.67 to ~0.49 on a team change — SumerSports).

Ascension κ **.** Driven by draft capital, breakout age, college dominator (decayed by NFL season), current route share vs projected route share, and vacated targets on the depth chart. Cap at 0.60.

4.4 Tight end

Curve. Peak 27.5 with an unusually **steep ramp** (`c_up = 0.030`), which encodes the documented rookie-to-sophomore leap (PFF, ESPN) and a **shallow decline**, matching the finding that TEs age better than other flex positions (FTN). In the worked example this produces a +73% five-year trajectory for a 24-year-old TE — the largest of any archetype — which is exactly the delayed-breakout behaviour you asked for.

TE premium. Handled in the scoring engine (per-reception bonus), which then flows through the replacement engine automatically. TEP raises both TE points *and* TE replacement level; the net value gain comes from the *shape* of the TE curve (steep top, shallow tail), not from the bonus itself. This is more principled than KTC-style TEP tiers applied as a post-hoc multiplier.

σ drivers. Route participation is the gatekeeper. A TE below ~65% route share should carry a large σ penalty; above it, YPRR becomes meaningfully predictive (Fantasy Life).

4.5 Interior defensive line (DT / iDL)

Curve. Latest-peaking IDP group (27.5) with a shallow decline — big-bodied interior players are less athleticism-dependent than edge or off-ball players.

Key drivers. Snap share and **pass-rush snap share** are the value gate; many productive real-life DTs are two-down players with no fantasy path. Sack totals should be regressed toward expected sacks (Fantasy Life IDP).

Format sensitivity. DT value is almost entirely a function of whether your league has a required DT slot or big tackle bonuses. The replacement engine handles this: in a combined "DL" group, DTs are compared against edge rushers and mostly lose; in a DT-required league, DT replacement level collapses and mid-tier DTs become valuable. **This must be configurable, not hard-coded.**

4.6 Edge rusher (DE / OLB / EDGE)

Curve. Peak 27, moderate decline.

Key drivers. Pass-rush snaps × pressure rate → expected sacks. **Regress projected sacks toward expected sacks** with weight ~0.35 for $\mu(t \geq 1)$ — this is the single highest-value IDP-specific adjustment available, and the sack leaderboard's documented instability supports it (PFF).

σ. Highest `event_volatility` of any position (sack-dependent scoring). In the worked example the elite EDGE carries `event_volatility = 0.55` , which widens his distribution without reducing his median.

Rotation risk. Snap share below ~70% should trigger a role-security penalty even for a high-pressure-rate player.

4.7 Off-ball linebacker

Curve. Peak 26.5 with a **steeper decline than the DL groups** — [Hypothesis] LB fantasy value is destroyed by scheme change and drafted competition more often than by physical decline, so the effective curve looks steeper even though the athlete may not be declining.

Key drivers, in order: snap share → three-down/coverage role → green-dot status → tackles vs expected. A two-down linebacker is nearly worthless in fantasy regardless of talent (Yahoo IDP explainer).

Lowest σ of any position (`CV_BASE = 0.26`) because tackle volume is the most stable IDP production stream. This gives three-down LBs strong contender value and is why the demo's green-dot LB scores 8,946 for a contender but 4,750 for a rebuilder.

Scarcity caution. LB is the deepest IDP group in tackle-heavy formats. High replacement level is the mechanism that stops the model from over-ranking LBs; see §2.7.

4.8 Cornerback — the position that needs its own logic

The answer to your question is **yes, corners need a modified model**, but the fix is narrow and specific.

The problem. Fantasy CB production is driven by *targets faced*, which is inversely related to coverage quality. A shutdown corner gets avoided; a weaker or slot corner accumulates tackles, breakups and picks (Yahoo, NFL.com).

The fix — three specific changes:

1. **Never use coverage grade as a positive input to** `μ` **.** Use `targets_faced_per_coverage_snap` instead. If you have no charting data, use team pass-defense DVOA/EPA rank as a crude proxy for how often the defense is thrown at, and slot-snap share as a proxy for target proximity.
2. **Add a rookie-CB opportunity bump and a decay.** Rookie corners are tested more and see that opportunity fade as they establish themselves. Model this as a *negative* `κ` (≈ −0.10) for high-draft-capital, high-coverage-grade corners: their fantasy role shrinks as their real-life quality is respected. This is one of the few places in sports modelling where being good makes you less valuable.
3. **Highest σ and highest base hazard.** `CV_BASE = 0.40` , `event_volatility` default 0.70 (INT-dependent scoring), and the highest hazard schedule of any position — DBs are the most volatile group year over year.

Pass breakups over interceptions in every regression target: breakups are the more repeatable signal.

4.9 Safety

Curve. Peak 27, moderate decline.

Archetype is everything. `box` / `slot` safeties get tackle volume and behave like mini-linebackers; `deep` safeties are big-play dependent with weak floors. Cover-2-heavy defenses keep safeties near the line; Cover-3/Cover-4 push them deep (Fantasy Life). Encode `box_snap_pct` directly into role security and σ.

Scheme-change risk is the dominant hazard driver — a new coordinator can convert a box safety into a deep safety and erase 40% of his fantasy production without any change in the player.

4.10 Position-designation risk (cross-cutting)

Eligibility flips (DE→LB, EDGE→DL, S→CB, hybrid LB/S) can move a player between a scarce group and a deep one, changing value more than any on-field factor. **[Recommendation]** Model it explicitly as a scenario, not a fudge:

```
DV = Σ_j  P(designation j)  · DV(player | designation j)
```

Store `designation_risk ∈ [0,1]` per player and compute value under each plausible designation using that group's replacement level. Show both numbers on the player card ("worth 6,600 as an LB, 3,900 as a DL — 25% chance of reclassification"). The demo's green-dot LB carries `designation_risk = 0.05` ; a 3-4 OLB who might be listed as either would carry 0.4–0.6.

Part 5 — Mathematical formulation

Every formula here is implemented in `dynasty_engine.py` ; function names are given so you can trace them.

5.1 Projection blending → `blend_projections()`

Given sources *k* = 1…K with per-game projections `f_k` , games `g_k` , and accuracy weights `w_k` :

```
trim the top and bottom ⌈0.10K⌉ sources by f_k when K ≥ 5
```

```
μ₀      = Σ w_k f_k / Σ w_k
G₀      = Σ w_k g_k / Σ w_k
σ_src²  = Σ w_k (f_k − μ₀)² / Σ w_k
```

`σ_src` is the **disagreement** component only, not the full outcome distribution.

Accuracy weights. For position *p* and source *k*, using only seasons strictly before the current one:

```
MASE_{k,p} = MAE_{k,p} / MAE_{naive,p}          naive = prior-season PPG carried forward
w_{k,p}    = max(0, 1 − MASE_{k,p})^2 · ρ^(years since evaluated)
```

with recency decay `ρ = 0.85`. Squaring sharpens separation between good and bad sources; the floor at zero drops sources worse than the naive baseline. This mirrors the accuracy-weighted approach that performs best in FFA's public comparisons (FFA).

Raw stats vs fantasy points — recommendation. Ingest raw stat lines and compute fantasy points internally, with projected fantasy points as a fallback. Reasons: (1) your league is Superflex/TEP/PPR/IDP and almost no public source publishes points in that exact configuration; (2) you need per-stat scoring to support arbitrary league configs, which is the whole point of the product; (3) raw stats let you compute TD-dependence and event-volatility, which points totals hide; (4) it lets you re-score history under today's settings for backtesting. Accept `proj_fpts` only when a source gives you nothing else, and flag those rows as `scoring_native = false`.

5.2 Scoring → `score_stats()`

```
FP = Σ_s  stat_s · coeff_s(league)          plus position conditionals (TE reception bonus)
```

5.3 Replacement level → `ReplacementEngine`

Let `T` = teams, `st_g` = fixed starters at lineup group *g*, and `F` = the set of flex slots, each with `n_f` slots per team and eligible groups `E_f`.

```
1. counts[g] ← T · st_g
2. for each flex slot f, repeat n_f · T times:
       g* ← argmax_{g ∈ E_f}  Pool_g(counts[g] + 1)
       counts[g*] ← counts[g*] + 1
3. replacement_rank[g] = counts[g] + round(T · buffer_g)
4. R_g = Pool_g(replacement_rank[g])
```

`Pool_g(r)` is the projected FPG of the *r*-th ranked player at group *g* under **this league's scoring**. The greedy flex allocation is the standard iterative solution and converges in one pass because the pools are monotone decreasing.

`buffer_g` is the waiver-depth parameter — how many players per team beyond the startable count are rostered *and* startable-quality. It is the single most important tuning knob in the whole model, because it sets the origin of the value scale.

Best-ball leagues: raise `buffer_g` by ~0.5 for RB/WR (deeper rosters) and use the expected *max* rather than mean when computing `Pool_g` from weekly distributions.

5.4 Aging → `aging_mult()`, `effective_age()`

```
A_p(a) = exp( −c_up,p · (peak_p − a)² )      a ≤ peak_p
         exp( −c_dn,p · (a − peak_p)² )      a >  peak_p
```

Asymmetric Gaussian: smooth, two interpretable parameters per side, easy to refit. **[Recommendation]** Use this rather than fixed multipliers or a lookup table for v1 — you can refit `(peak, c_up, c_dn)` per position with three-parameter least squares on delta-method year-pairs, and it degrades gracefully with small IDP samples. Upgrade to a monotone GAM only if backtesting shows systematic curvature error.

Mileage:

```
a_eff = a + min( cap_p , max(0, (L − L_ref,p) / L_unit,p) )
```

where `L` is career load in the position's unit (RB: touches; WR/TE: targets; QB: dropbacks; IDP: defensive snaps). Applied on both sides but only ever increases effective age.

Trajectory:

```
μ(t) = μ₀ · [ A_p(a_eff(t)) / A_p(a_eff(0)) ] · Asc(t)
Asc(t) = 1 + κ · (1 − e^(−t/τ)),   τ = 1.5,  κ ∈ [−0.15, 0.60]
```

Ascension coefficient:

$$\kappa = \text{clip}(\beta_1 \cdot DC + \beta_2 \cdot w\_col(n) \cdot COL + \beta_3 \cdot OPP + \beta_4 \cdot YOUTH - \beta_5 \cdot INCUMBENT , -0.15, 0.60)$$

where DC = draft-capital score (0–1), COL = standardized college production, $w\_col(n) = \exp(-1.1(n-1))$, OPP = vacated opportunity share on the depth chart, $YOUTH = 1$ if $nfl\_season \le 2$, $INCUMBENT$ = current role saturation (a player already at 95% snap share cannot ascend). κ is forced to 0 for $nfl\_season \ge 4$ except for documented role changes.

5.5 Survival → `hazard()`, `S(t)`

Discrete-time hazard, base rate by position and age band, multiplicatively adjusted:

$$h(t) = \text{clip}(h\_base,p(a\_t) \cdot \Pi\_j \ (1 + \gamma\_j \cdot (ref\_j - x\_j)) , 0.02, 0.85)$$
$$S(t) = \Pi\_{k=1..t} \ (1 - h(k)), \qquad S(0) = 1$$

with adjusters for role security (γ = 0.55), draft capital × decay (0.30), contract security (0.25), durability (0.35), production z-score (–0.18), and IDP designation risk (0.20).

Expected remaining fantasy-relevant seasons — a directly displayable output:

$$E[remaining] = \Sigma\_{t \ge 1} \ S(t)$$

5.6 Uncertainty → `sigma_fpg()`

$$CV\_i = CV\_base,p \cdot (1 + 0.90 \cdot role\_unc) \cdot (1 + 0.45 \cdot event\_vol) \cdot (1 + 0.60 \cdot small\_sample)$$
$$\sigma(0) = \text{sqrt}((CV\_i \cdot \mu_0)^2 + \sigma\_src^2)$$
$$\sigma(t) = \text{sqrt}(\sigma(0)^2 + (\psi\_p \cdot \mu(t) \cdot \sqrt{t})^2)$$

ψ_p is per-year drift volatility — how fast the world changes for that position. The $\sqrt{t}$ growth is the standard random-walk assumption and is why long-horizon values have wide bands.

5.7 Season value → `season_starter_value()`

$$z = (\mu(t) - R\_p) / \sigma(t)$$
$$SSV(t) = [(\mu(t) - R\_p) \cdot \Phi(z) + \sigma(t) \cdot \varphi(z)] \cdot G(t)$$

5.8 Discounted dynasty value → `DynastyModel.value()`

$$DV\_s = \sum_{t=0}^{H} u\_s(t) \cdot d\_s\hat{}t \cdot S(t) \cdot SSV(t)$$

$$\Psi = \text{sqrt}(\Sigma\_t [d\_s\hat{}t \cdot \sigma(t) \cdot G(t) \cdot S(t)]^2) \qquad \text{path dispersion}$$
$$DV\_s^* = \max(0, DV\_s - \lambda\_s \cdot 0.35 \cdot \Psi) \qquad \text{certainty equivalent}$$

5.9 Trade-value conversion → `to_trade_value()`

$$TV\_s(i) = 10000 \cdot (DV^*\_s(i) / anchor\_s)\hat{}\gamma , \qquad \gamma = 1.20$$
$$anchor\_s = \max\_i \ DV^*\_s(i) / (target/10000)\hat{}(1/\gamma)$$

$\gamma > 1$ creates the **stud premium**: value is convex in surplus because starting slots are finite. Each strategy profile gets its own anchor, so values are comparable **within** a profile. When evaluating a trade between two managers with different timelines, **evaluate the trade twice — once in each manager's currency** — and look for deals that are positive in both. That "double-positive" search is the core of a good trade finder and is impossible with a single universal number.

5.10 Market layer

Blend (Bayesian-style precision weighting):

$$\tau\_model = 0.85 \cdot (1 - 0.4 \cdot small\_sample)$$
$$\tau\_market = 1.00 \cdot m\_pos \cdot m\_exp \cdot (1 - 0.5 \cdot mkt\_dispersion)$$

```
      m_pos = 0.45 for IDP groups, 1.0 for offense       (thin IDP market)
      m_exp = 0.55 for rookies, 1.0 otherwise           (hype-driven rookie market)
w        = τ_model / (τ_model + τ_market)
Blended  = w·TV_model + (1−w)·TV_market
```

Market gap and realizable alpha:

```
Gap      = TV_model − TV_market
Gap%     = Gap / TV_market
Liquidity= clip( 0.35 + 1.6·mkt_dispersion + 0.3·trade_frequency_z , 0.2, 1.0 )
Alpha    = Gap · Liquidity
```

The liquidity term is your answer to "theoretical value isn't obtainable." A +2,000 gap on a player nobody disagrees about is unrealizable; a +1,200 gap on a player with wide market dispersion is a live trade target. **Rank buy/sell candidates by Alpha, not by Gap.**

Momentum guard [Recommendation]: suppress buy signals when `mkt_momentum_30d` is strongly negative and the model gap is driven by a stale projection — i.e., require that the gap has persisted for ≥ 14 days before surfacing it. This prevents the tool from telling users to buy players in the middle of bad news the projection hasn't absorbed yet.

5.11 Package / trade math → `package_value()`

```
Package(v₁..vₙ) = ( Σ vᵢ^θ )^(1/θ)  −  C_spot · max(0, n − spots_free)
```

`θ = 1.20` [Prior]. At θ = 1 this is naive addition. At θ > 1 the best asset dominates, so a 3-for-1 must clear a real bar. In the worked example, a veteran WR (2,485) plus a young TE (6,367) sums naively to 8,852 but packages to **8,041** against a 9,800 single asset — the consolidation premium is ~1,760 points, or about 10% of the trade.

Roster-context multipliers (applied to each side after packaging):

```
Need_g   = 1 + η · (starters_g − startable_count_g) / starters_g,   η = 0.12, clipped to [0.85, 1.25]
Timeline = evaluate with the receiving team's own strategy profile
```

5.12 Rookie picks → `pick_value()`

Treat a pick as a random variable over outcomes:

```
V(pick) = [ P_hit·V_hit + P_mid·V_mid + P_miss·V_miss ] · Class · d_s^(years_out) · Flex_s
```

`Class ∈ [0.85, 1.15]` for draft-class strength; `Flex_s` = 0.92 / 1.00 / 1.08 for contender / balanced / rebuilder (picks are more useful to a rebuilder: they are cheap, roster-flexible, and liquid). Also report `P_hit`, ceiling and median separately — a pick is a distribution, and the card should say so.

5.13 Rest-of-season value

```
μ_ros   = w_proj·μ_preseason_updated + (1 − w_proj)·μ_recent_form
w_proj  = n_prior / (n_prior + weeks_played),     n_prior ≈ 6 for offense, 8 for IDP
SSV_ros = Σ_{w = now..end} 1{not bye} · P(active in week w) · E[max(0, μ_ros − R_p)] · PlayoffWeight(w)
PlayoffWeight(w) = 1 + π·1{w ∈ league playoff weeks},   π = 0.35 contender, 0 rebuilder
```

`n_prior` is a shrinkage constant: at week 6, roughly half the weight is still on the preseason projection. Injured players use expected return week and a ramp-up discount for the first two weeks back.

How ROS differs from dynasty value. ROS is a *single* horizon with no survival term, no aging term, no discounting, and playoff-week weighting. A contender's ROS ranking should also be **replacement-adjusted against that manager's own bench**, not against league-wide waivers — the relevant baseline is who they'd actually start instead.

Part 6 — Weight recommendations

Every number in this part is a starting prior. None of it is validated. Ship it, backtest it, and expect 30–50% of these to move.

6.1 Aging curve parameters `(peak, c_up, c_dn)`

Group	peak	c_up	c_dn	Implied: value at 23	at 27	at 30	at 33

QB pocket	30.0	0.006	0.010	0.75	0.95	1.00	0.91
QB dual	26.0	0.010	0.028	0.91	0.97	0.67	0.32
RB	25.0	0.012	0.030	0.95	0.89	0.47	0.11
WR	27.0	0.014	0.024	0.80	1.00	0.81	0.44
TE	27.5	0.030	0.018	0.51	0.99	0.90	0.61
DT	27.5	0.016	0.020	0.72	0.996	0.88	0.60
EDGE	27.0	0.014	0.022	0.80	1.00	0.82	0.47
LB	26.5	0.012	0.030	0.86	0.99	0.68	0.26
CB	26.0	0.010	0.030	0.91	0.97	0.65	0.24
S	27.0	0.012	0.026	0.83	1.00	0.79	0.40

6.2 Mileage parameters

Position	Unit	Reference	Per effective year	Cap
RB	career touches	750	500	2.5 yrs
WR	career targets	500	450	1.5
TE	career targets	400	450	1.5
QB	career dropbacks	2000	2500	1.0
LB / EDGE / CB / S	defensive snaps	3000	2200–2400	1.5
DT	defensive snaps	3000	2400	1.0

6.3 Base annual hazard (probability of losing fantasy-startable status)

Position	≤24	25–26	27–28	29–30	31–32	33+
QB	0.16	0.10	0.10	0.09	0.14	0.24→0.38
RB	0.11	0.15→0.22	0.30→0.38	0.50	0.50	0.50
WR	0.13	0.09	0.09→0.13	0.13→0.22	0.22→0.34	0.46
TE	0.16	0.10	0.10	0.12	0.20	0.32→0.44
DT	0.14	0.10	0.10	0.14	0.24	0.38
EDGE	0.14	0.10	0.10	0.15	0.26	0.40
LB	0.15	0.12	0.12→0.18	0.18→0.28	0.28	0.42
CB	0.18	0.15	0.15→0.22	0.22→0.32	0.32	0.46
S	0.16	0.13	0.13→0.19	0.19→0.29	0.29	0.44

Calibration target: the average $S(3)$ across your rostered-player population should match the observed 3-year startable-retention rate in your historical database, per position. This is the first thing to fit and the easiest to validate.

6.4 Uncertainty parameters

Position	cv_{base} (seasonal PPG dispersion)	ψ (per-year drift vol)
QB	0.26	0.16
RB	0.36	0.30

WR	0.34	0.22
TE	0.36	0.22
DT	0.32	0.20
EDGE	0.34	0.22
LB	0.26	0.22
CB	0.40	0.30
S	0.30	0.24

LB carries the lowest CV (tackle volume is stable); CB the highest (event-dependent, role-volatile).

6.5 Strategy profiles

Profile	discount d	horizon H	usability $u(0..)$	risk λ	pick premium
Contender	0.72	4	1.00, 0.95, 0.80, 0.65, 0.55	+0.20 (averse)	0.92
Balanced	0.85	6	1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80	+0.10	1.00
Rebuilder	0.93	8	**0.45**, 0.75, 1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80	−0.10 (seeking)	1.08

On the discount rate. A balanced $d = 0.85$ implies a ~17.6% annual required return on future fantasy production — well above a financial discount rate, and deliberately so. Three justifications: (1) genuine forecast uncertainty compounds; (2) roster spots have carrying cost; (3) dynasty leagues themselves have a survival rate — leagues fold, managers quit, and rules change. The $u(0) = 0.45$ for rebuilders is the more important parameter: it says **a rebuilder gets less than half the benefit from this season's points**, which is what actually drives contender/rebuilder divergence, more than the discount rate does.

6.6 Other parameters

Parameter	Prior	Notes
γ stud-premium exponent	1.20	Calibrate so your model's top-30 spacing matches market spacing
θ package CES exponent	1.20	Raise toward 1.35 in shallow-roster leagues
C_{spot} roster-spot cost	120 pts	≈1.2% of a max-value asset
Draft-capital decay	$\exp(-0.45(n-1))$	~10% remaining by year 6
College decay	$\exp(-1.1(n-1))$	~0 by year 4
Sack→expected-sack regression	0.35	EDGE/DT only, applied to $\mu(t\geq 1)$
TD-rate regression	0.40	Offense, applied to $\mu(t\geq 1)$
ROS shrinkage n_{prior}	6 (off) / 8 (IDP)	Weeks of prior-equivalent weight
Playoff weight π	0.35 contender / 0 rebuilder	ROS only

6.7 Explainability weights (illustrative, from the actual model run)

The decomposition is **computed, not assigned**. Horizon shares are exact (they are additive terms of the sum); factor attributions are log-scale contributions relative to a neutral baseline player. Sample output:

Archetype	yr 0	yr 1–2	yr 3+	survival drag	5-yr trajectory
Young elite QB (dual, 24)	27%	44%	29%	10%	−26%
Aging QB (pocket, 34)	53%	41%	7%	25%	−53%
Young RB (23)	27%	48%	25%	24%	−37%
Aging RB (29, 1720 touches)	85%	15%	0%	15%	−95%

Young ascending WR (23)	17%	40%	43%	21%	**+52%**
Veteran WR (30)	63%	34%	2%	17%	–83%
Young TE (24)	14%	40%	46%	24%	**+73%**
Rookie WR (22, 1st-rd)	13%	38%	49%	29%	**+117%**
Three-down LB (25)	31%	47%	22%	18%	–43%

Part 7 — Model alternatives

7.1 Three versions compared

	A — Transparent parametric (recommended v1)	**B — Intermediate statistical**	**C — Advanced ML**
Core	Closed-form curves + hazard + option value	A, plus fitted GAM/monotone-GBM for `μ(t)` residual and a Cox/discrete-time hazard for `S(t)`	End-to-end GBM/NN predicting 3-yr discounted VOR directly
Data need	Projections + basic history	+ 10 yrs of clean feature snapshots	+ everything, cleanly versioned, plus PFF/SIS charting
Explainability	Complete — every term nameable	Good — shape plots + SHAP	Poor without heavy tooling
IDP small samples	Handles fine (parameters shared across positions, only 3 free params per curve)	Needs hierarchical pooling	Overfits DT/CB badly
Overfit risk	Low	Medium	High
Time to ship	3–5 weeks	+6–10 weeks	+3 months
Expected accuracy gain	baseline	**+5–12% rank correlation** [Hypothesis]	+0–6% over B, and often negative on IDP
Maintenance	Edit a parameter table	Retrain annually	Retrain + monitor drift + debug

7.2 Why not go straight to gradient boosting

1. **Your effective sample is small.** One player-season per row, ~700 fantasy-relevant offensive players and ~400 IDP per year, and a target that requires 3+ years of follow-up. You have on the order of 3,000–6,000 usable rows *with complete multi-year outcomes* — and they are heavily correlated within player.
2. **Your target is partly constructed.** Discounted future VOR depends on your own replacement levels and scoring config. A model trained to predict a constructed target can only learn the construction.
3. **The structure is known.** Aging, survival and truncation-at-replacement are not things you should make a tree discover; they are things you should impose. Trees spend their capacity rediscovering `age` and `snap share`.
4. **You need explanations on a player card.** That is a product requirement, not a modelling preference.
5. **The public evidence for complex projection models beating good simple blends is weak.** Published fantasy-projection ML work reports errors in the same range as expert consensus (arXiv 1505.06918, arXiv 2111.02874).

Where ML genuinely helps, and where I'd add it in Phase 8: (a) predicting `κ` — next-season *role* change (snap/route/target share) from current usage, depth chart and transactions; that's a well-posed one-year classification/regression problem with plenty of rows; (b) predicting games missed; (c) predicting the *hazard* of losing a starting job. All three are sub-models with clean targets, and all three plug into the parametric skeleton without breaking explainability.

Part 8 — Backtesting plan

8.1 The cardinal rule: as-of snapshots

Build an **`as_of` dimension into every fact table**. A backtest row for season *Y* may only use rows with `as_of_date ≤ Aug 15 of year Y`. No

exceptions, including:

- Projection files must be the *preseason* version, archived at ingest time. If you only have current-version projections, you cannot backtest projections — you must simulate them (see 8.3).
- Market values must be the snapshot from that date, not today's.
- Source accuracy weights must be trained on seasons `< Y`.
- Position designation must be the designation *as of that date*, not the corrected one.
- Injury/roster status must be the report from that week.

Start archiving today, even before the model is built. A daily snapshot of your 10-source scraper plus weekly nflverse pulls costs almost nothing and is the only way you will ever have a market-appreciation backtest. This is the single highest-value thing you can do this week.

8.2 Targets

Target	Definition	Model output tested	Metric
T1	Next-season fantasy PPG	`μ(1)`	MAE, RMSE, Spearman by position
T2	Next-season VOR (season total)	`SSV(1)`	Spearman, NDCG@24
T3	3-year discounted VOR (realised)	`DV_balanced`	Spearman, top-k precision
T4	Top-12/24/36 finish next season	`P(elite)`, `P(>repl)`	Brier, log loss, calibration curve
T5	Still a startable starter in 3 years	$S(3)\cdot\Phi(z_3)$	Brier, calibration
T6	Market value 12 months later	`Gap`, `Alpha`	Spearman of Gap vs realised market change; P&L of a simulated buy/sell portfolio
T7	Breakout (≥35% PPG increase)	`P(breakout)`	Precision/recall @ threshold, PR-AUC
T8	Value collapse within 1 year	`P(collapse)`	Brier, recall at 20% flag rate

T6 is the commercially important one. A model that predicts football but not the market cannot generate profitable trades. A model that predicts the market but not football is just a lagging copy of KTC. You need both, measured separately.

8.3 Handling the "I don't have historical projections" problem

You almost certainly lack archived preseason projections for 2015–2024. **[Recommendation]** Build a **reconstructed projection baseline** and be explicit that it is a proxy:

```
μ̂₀(Y) = shrink( 3-yr weighted PPG through Y-1 , positional mean , k=0.35·(1/games) )
         adjusted by depth-chart-derived role change and a rookie prior from draft slot
```

This is a *worse* projection than a real preseason consensus, which means your backtest understates the final model's accuracy — an acceptable bias direction. Validate the proxy on the 2–3 seasons where you *do* have archived projections, measure the gap, and report all historical results with that caveat attached.

8.4 Validation scheme

Rolling-origin, expanding window, by season:

```
fold 1: train ≤2016  validate 2017      test 2018
fold 2: train ≤2017  validate 2018      test 2019
...
fold k: train ≤2022  validate 2023      test 2024
final:  train ≤2024  →  live 2025-26 forward test
```

Never shuffle rows across seasons. Never split within a season. Group by `player_id` so the same player never straddles train and test within a fold when evaluating multi-year targets.

8.5 Baselines the model must beat

Baseline	Description	If the model doesn't beat it

B0	Prior-season PPG carried forward	Something is badly wrong
B1	Blended projection alone (no dynasty layer)	The whole dynasty layer is noise — ship B1
B2	Projection × linear age penalty (e.g. −4%/yr past 26)	Ship B2; your curves aren't earning their complexity
B3	**Market consensus (KTC/FantasyCalc blend)**	You have a market copy, not a model
B4	Projection + age + a single position dummy	Your IDP-specific work isn't paying

The specific bar I'd set: the fundamental model must beat B3 on **T3 (3-year realised VOR, Spearman) by ≥0.03** and on **T6 (market-change prediction) with statistically significant positive Spearman**, evaluated separately for offense and IDP. If it beats B3 on offense but not IDP, ship it with the market weighted higher for offense and the model weighted higher for IDP — which, per §5.10, is already the default direction.

8.6 Per-position evaluation and the complexity ratchet

Evaluate **every metric separately by position**, and require each position-specific complication to earn its place:

- Does the archetype split (`QB_pocket` vs `QB_dual`) improve QB Spearman vs a single QB curve? If not, drop it.
- Does the mileage term improve RB T3 vs age alone? If not, drop it.
- Does the CB-specific target-faced logic improve CB T2 vs the generic model? If not, collapse CB into a generic DB model.
- Does `E[max(0, μ−R)]` beat plain `μ−R` on T3 and on T4 calibration? *This is the most important single ablation in the whole plan* — the option formulation is my strongest design claim, and it should be the first thing you try to falsify.

Log every ablation result in a `model_experiments` table with the fold-level metrics. **[Recommendation]** Adopt a hard rule: no parameter enters production without a logged ablation showing improvement on at least two of three folds.

8.7 Calibration checks

For every probability output, plot predicted vs observed in deciles and report the calibration slope/intercept. A `P(breakout) = 0.30` bucket must break out about 30% of the time. Uncalibrated probabilities on a player card are worse than no probabilities, because users anchor on them.

Part 9 — Implementation blueprint

9.1 Recommended stack

Given what you already run (Python/Playwright scrapers, single-file HTML/JS dashboards, Hetzner VPS, PowerShell deploys), the lowest-friction path is:

Layer	Choice	Why
Storage	**PostgreSQL 16** on the VPS	You need `as_of` snapshots, joins across 10 market sources, and window functions for ranking. SQLite will hurt once you have 10 years × weekly snapshots.
Compute	**Python 3.11**, pandas + numpy + scipy; `nflreadpy` or `nfl_data_py` for nflverse	`dynasty_engine.py` is stdlib-only so it can also run in a Lambda/cron with zero install
Scheduling	systemd timers or cron on the VPS	No new infra
Serving	**FastAPI** exposing `/values` , `/player/{id}` , `/trade` , `/roster` + a nightly static JSON export	Static export keeps your existing single-file dashboard pattern working; the API is there when you need live league sync
Config	JSON league configs in Postgres, hashed → `config_id`	Every stored value must be traceable to a league config *and* a model version
Versioning	`model_version` (semver) + `param_set_id` + `config_id` on every value row	Non-negotiable; see 9.6

Data sources: **nflverse** covers an astonishing amount for free — snap counts, contracts (from OverTheCap), draft picks, injuries, depth charts, Next Gen Stats, and expected fantasy points via `load_ff_opportunity` (nflreadr reference, nflreadpy). Start there; add PFF/SIS charting only when you've proven you need pressure rate and alignment data.

9.2 Schema

Full DDL is in `schema.sql`. Core tables:

```
players                 identity, DOB, draft, true position, archetype
player_seasons          per-season NFL stats + snap/role, as_of-versioned
projections_raw         one row per (source, player, season, as_of) — raw stat line JSON
projection_blend        blended μ₀, σ_src, G₀ per (player, season, config, as_of)
league_configs          JSON scoring + lineup + buffers, hashed
replacement_levels      R by (config, group, season, as_of)
model_params            versioned parameter sets (curves, hazards, CVs, strategy profiles)
player_values           the output: DV/TV per (player, config, strategy, model_version, as_of)
value_paths             per-season μ, σ, SSV, S(t) for the career chart
market_values           per (source, player, format, as_of) + type tag
market_gaps             gap, liquidity, alpha per (player, config, as_of)
news_events             structured event adjustments with decay
rookie_picks            pick outcome distributions by slot/format/class
model_experiments       ablation + backtest results
```

9.3 Structured news-event system

This is the mechanism that replaces arbitrary manual bumps. Every event is a row:

```
{
  "event_id": "evt_2026_08_02_0031",
  "player_id": "00-0036355",
  "event_type": "DEPTH_CHART_COMPETITION_ADDED",
  "effective_date": "2026-08-02",
  "confidence": 0.65,
  "source_reliability": 0.80,
  "already_in_projection": false,
  "impact": { "mu_pct": -0.08, "role_security_delta": -0.15,
              "sigma_mult": 1.20, "hazard_mult": 1.10 },
  "duration_days": 120,
  "decay": "exponential",
  "half_life_days": 45,
  "notes": "Team signed veteran at same position"
}
```

Applied as: `effect(t) = impact × confidence × source_reliability × exp(−ln2 · days_since / half_life)`, and **skipped entirely if `already_in_projection = true`**. Event types should be a closed enum with default impact magnitudes stored in `model_params`, so a human sets *type* and *confidence*, never a raw number.

Confirmed vs speculative: `confidence` ≥ 0.9 = officially confirmed (transaction wire, team announcement); 0.5–0.8 = credible beat reporting; < 0.5 = speculation, which should adjust **σ only**, never μ. That rule alone prevents most rumor-driven value swings.

9.4 Pipeline and cadence

Job	Cadence	What it does
`ingest_nflverse`	Daily (in-season), weekly (off)	rosters, snaps, injuries, depth charts, contracts, stats
`ingest_projections`	Weekly (off), Tue+Fri (in-season)	pull each source, store raw with `as_of`
`ingest_market`	Daily	your 10-source scraper, tagged by market type
`blend_projections`	After ingest	weighted trimmed mean per position
`compute_replacement`	After blend, per config	flex allocation + buffers
`compute_values`	After replacement	the full engine, all configs × strategies
`compute_market_gaps`	After values + market	gap, liquidity, alpha, buy/sell flags
`export_static`	After values	JSON bundles for the dashboard
`refit_params`	Annually (Feb–Apr)	aging curves, hazards, source weights, pick distributions
`backtest`	Annually + on any param change	rolling-origin validation, writes to `model_experiments`

In-season weekly update touches: projections, market, news events, injury status, ROS module. It does **not** refit curves — parameter refits happen

only in the offseason, so mid-season values move for reasons users can understand.

9.5 Missing-data rules

Explicit, deterministic, and surfaced to the user as a data-quality score:

Missing	Rule
Projection for a rostered player	Impute from prior-season PPG × position aging × 0.85 role haircut; set `σ` ×1.5; flag `imputed=true`
Games projection	Default 16.0 offense, 16.0 IDP; adjust by durability
Career load (mileage)	Estimate from games × position-median load per game
Snap share	Use depth-chart rank → prior snap share by rank and position
Contract data	Assume 2 years remaining, `contract_security = 0.5`
College production	`κ` contribution = 0; do not penalise
Market value	Omit gap/alpha entirely rather than imputing — never show a fabricated gap
Archetype	Use position default (`pocket` for QB, `box` for S, `boundary` for CB) and widen σ by 10%

Rule of thumb: missing data should always increase σ and never silently change μ.

9.6 Versioning

- `model_version` = semver of the *code*.
- `param_set_id` = FK to a frozen parameter row set. Changing a single aging-curve constant creates a new `param_set_id`.
- Every `player_values` row stores `(model_version, param_set_id, config_id, as_of)`.
- Never update a value row in place — insert a new one. Value history *is* the product (career-value charts, market-gap tracking, "what changed this week" diffs).
- Tag each released parameter set in git with the backtest report that justified it.

9.7 API contract

Request (`POST /values`) and **response** shapes are in `player_payload.example.json`. Summary:

```
// request
{ "league_config_id": "brisket_sf_tep_idp_12",
  "players": [ { "player_id": "...", "projections": [
      { "source": "SourceA", "stat_line": {"rec": 82, "rec_yd": 1120, "rec_td": 7}, "games": 16 },
      { "source": "SourceB", "fpts": 248.5, "games": 16 } ] } ],
  "strategies": ["contender","balanced","rebuilder"],
  "include": ["path","probs","explain","market"] }

// response (per player)
{ "player_id": "...", "pos": "WR", "age": 23.4,
  "projection": { "fpg": 14.0, "games": 16.0, "sigma_source": 1.4 },
  "replacement": { "group": "WR", "fpg": 6.15 },
  "values": { "contender": 9693, "balanced": 9800, "rebuilder": 9800 },
  "market": { "consensus": 7900, "dispersion": 0.26, "gap": 1900,
              "liquidity": 0.77, "alpha": 1463, "signal": "BUY" },
  "probs": { "above_replacement": 0.88, "elite_season": 0.33,
             "starter_in_3y": 0.73, "breakout": 0.45, "collapse_1y": 0.16 },
  "path": [ {"t":0,"age":23,"mu":14.0,"sigma":4.9,"ssv":128.4,"survival":1.00}, ... ],
  "explain": { "year_0_share":0.17, "years_1_2_share":0.40, "years_3_plus_share":0.43,
               "survival_drag":0.21, "traj_5y":0.52 },
  "meta": { "model_version":"1.0.0", "param_set_id":42, "as_of":"2026-07-27", "imputed":false } }
```

9.8 Pseudocode — the full nightly run

```
cfg   = load_league_config(config_id)
proj  = {}
```

```
for p in active_players():
    srcs = load_projection_sources(p, season, as_of=today)
    srcs = [s for s in srcs if s.weight > 0]
    proj[p.id] = blend_projections(srcs)                    # μ₀, σ_src, G₀

pool  = build_rank_curves(proj, cfg)                        # group -> FPG by rank
repl  = ReplacementEngine(cfg, pool)

model = DynastyModel(cfg, repl, pool=pool, stud_gamma=params.gamma)
model.calibrate(all_players, target=9800)                   # anchor per strategy

for p in active_players():
    pl  = assemble_player(p, proj[p.id], params)            # risk, archetype, mileage, κ
    pl  = apply_news_events(pl, load_events(p, today))       # skip if already_in_projection
    val = model.value(pl)
    mkt = load_market(p, cfg.format, as_of=today)
    if mkt:
        val.blended = model.blend_market(pl, val.trade_value["balanced"])
        val.gap     = val.trade_value["balanced"] - mkt.consensus
        val.alpha   = val.gap * liquidity(mkt.dispersion, mkt.trade_freq)
    persist(val, model_version, param_set_id, config_id, today)

export_static_bundles()
```

9.9 Error handling

- **Fail loudly on config errors** (unknown position, lineup slots exceeding roster size) — these silently corrupt every value in the league.
- **Fail soft on data errors** — a missing projection produces an imputed value with a data-quality flag, never a crash and never a zero.
- **Sanity gates before publishing a run:** no more than 5% of players may move >20% in trade value in a single non-event day; positional value totals must stay within ±10% of the prior run; replacement levels must be monotone in league size. Violations halt the export and page you.
- Keep the prior run's static bundle so a bad run can be rolled back with one command.

Part 10 — Worked examples

These numbers were produced by running `run_examples.py` against `dynasty_engine.py`. They are not illustrative hand-math. The 13 players are **hypothetical archetypes**, and the player pool used to derive replacement levels is a **synthetic rank-decay curve** calibrated to plausible 12-team Superflex / TE-premium / PPR / balanced-IDP scoring. Swap in your real projection file and every number recomputes.

10.1 League config and replacement levels

12 teams · QB1 RB2 WR3 TE1 FLEX1 SUPERFLEX1 · DL2 LB3 DB3 IDP_FLEX1 · 35-man rosters

Group	Startable slots league-wide	Replacement FPG
QB	24	**12.40**
RB	33	6.35
WR	36	6.15
TE	15	6.80
DL	24	6.70
LB	48	**7.32**
DB	36	6.17

Note the Superflex effect: QB replacement is nearly double every other group's, which is why raw QB points are so misleading.

10.2 The 13 archetypes

Player	Pos	Age	FPG	Surplus/g	Contender	Balanced	Rebuilder	Market	Gap	Blended

Young elite QB (dual)	QB	24	22.5	10.10	**9800**	7974	6265	9400	−1426	8723
Aging productive QB (pocket)	QB	34	19.0	6.60	3458	2112	1128	3600	−1488	2856
Young RB, three-down	RB	23	15.5	9.15	8492	6615	4982	7400	−785	7017
Aging RB, heavy mileage	RB	29	14.5	8.15	2674	1406	520	2200	−794	1798
Young ascending WR	WR	23	14.0	7.85	9693	**9800**	**9800**	7900	**+1900**	8839
Veteran WR, stable role	WR	30	15.5	9.35	4368	2485	1144	4100	−1615	3297
Young TE, year-2 leap	TE	24	10.5	3.70	5875	6367	6782	5600	+767	5983
Elite EDGE rusher	EDGE	26	14.5	7.80	6301	4712	3387	6200	−1488	5164
Three-down LB, green dot	LB	25	17.5	10.18	8946	6661	4750	5400	**+1261**	6267
Volatile boundary CB	CB	26	9.0	2.83	2055	1328	786	1800	−472	1465
Box safety, tackle floor	S	27	11.5	5.33	3239	2087	1228	2600	−513	2258
Rookie WR, 1st-rd capital	WR	22	9.0	2.85	6160	7051	7958	6800	+251	6935
Late-round breakout WR	WR	25	13.0	6.85	6140	4857	3719	3900	**+957**	4396

Read the strategy columns as three different currencies. Within each column, values are comparable and each column's top asset is normalised to ~9,800. Across columns, note the *ordering changes*: the elite QB is the #1 contender asset and only the #4 rebuilder asset; the rookie WR is #6 for a contender and #3 for a rebuilder; the aging RB is a usable contender piece (2,674) and nearly worthless in a rebuild (520).

10.3 Probability outputs

Player	P(>repl)	P(elite season)	P(starter in 3y)	P(breakout)	P(collapse ≤1y)
Young elite QB	0.92	0.54	0.75	0.22	0.13
Aging QB	0.86	0.33	0.24	0.08	0.35
Young RB	0.89	0.40	0.58	0.36	0.21
Aging RB	0.87	0.35	**0.01**	0.01	**0.65**
Young ascending WR	0.88	0.33	0.73	0.45	0.16
Veteran WR	0.92	0.42	0.20	0.05	0.35
Young TE	0.75	0.34	0.66	0.51	0.24
Elite EDGE	0.88	0.54	0.61	0.27	0.21
Three-down LB	**0.97**	0.51	0.66	0.24	**0.12**
Volatile CB	0.67	0.28	0.23	0.30	0.48
Box safety	0.85	0.41	0.35	0.21	0.30
Rookie WR	0.63	0.18	0.55	**0.52**	0.38
Late-round breakout WR	0.84	0.29	0.57	0.34	0.23

The aging RB is the clearest illustration of why dynasty ≠ projections: he has an **87% chance of being above replacement this season** and a **1% chance of being a startable asset in three years**.

10.4 Season-by-season paths (the career-value chart data)

Young RB, age 23, 310 career touches

t	age	age_eff	μ FPG	σ	SSV	S(t)	E[value]
0	23	23.0	15.50	7.41	147.9	1.00	147.9

1	24	24.0	17.48	9.85	185.0	0.91	167.8
2	25	25.2	18.40	11.74	204.2	0.82	167.9
3	26	26.7	17.27	12.18	191.1	0.71	135.6
4	27	28.2	13.96	10.73	143.5	0.58	82.8
5	28	29.7	9.78	8.10	82.4	0.44	36.3
6	29	31.2	5.95	5.31	30.3	0.30	9.2

Watch `age_eff` outrun `age` as he accumulates touches — by t=4 he is a 27-year-old with the effective profile of a 28.2-year-old. This is the mileage mechanism doing exactly what you asked for: two 25-year-old backs with different workloads get different values.

Aging RB, age 29, 1,720 career touches

t	age	age_eff	μ FPG	σ	SSV	S(t)	E[value]
0	29	30.9	14.50	7.25	129.4	1.00	129.4
1	30	32.5	7.87	4.72	42.2	0.56	23.8
2	31	33.5	4.78	3.37	10.8	0.32	3.4
3	32	34.5	2.79	2.38	1.1	0.18	0.2

He enters the model already at effective age 30.9. Note that 85% of his entire balanced value comes from season 0 — which is precisely why he is a legitimate contender asset and a rebuilder should not own him at any price above scrap.

10.5 The scarcity demonstration (your linebacker/safety requirement)

```
LB replacement 7.32 FPG   |   DB replacement 6.17 FPG

Three-down LB @ 17.5 FPG  →  surplus 10.18  →  trade value 6,661
Box safety    @ 11.5 FPG  →  surplus  5.33  →  trade value 2,087
Mid-tier LB   @ 12.5 FPG  →  surplus  5.18  →  worth LESS than the safety
```

A linebacker scoring a full point per game more than the safety is worth **less**, because the 42nd-best linebacker is nearly as good as him and the 40th-best defensive back is not. No hand-tuned positional multiplier was needed — this falls out of the replacement engine.

10.6 Trade math — 2-for-1 consolidation

```
Side A: young ascending WR                      9,800
Side B: veteran WR (2,485) + young TE (6,367)   naive sum 8,852
        CES package value (θ=1.20)                        8,041
        → consolidation premium to A: +1,759 (+9.9% of the trade)

Same trade, receiving team has only 1 free roster spot:
        Side B = 7,921   (roster-spot charge applied)
```

The two-for-one is **not** judged by adding values. And because each manager is evaluated in their own currency, the trade finder can identify deals that are positive for *both* sides — a contender giving up the young TE for the veteran WR gains in contender currency while the rebuilder gains in rebuilder currency. Those double-positive trades are the ones that actually get accepted.

10.7 Rookie picks

Pick	Contender	Balanced	Rebuilder	P(hit)	Ceiling
1.01	4,585	4,984	5,383	0.62	7,200
1.03	3,201	3,479	3,757	0.50	5,900
1.06	2,226	2,420	2,614	0.38	5,000
1.12	1,455	1,582	1,709	0.27	4,200
2.04	997	1,084	1,171	0.20	3,600

2.12	598	650	702	0.13	3,000
3.12	283	308	333	0.07	2,400
Next year's ~1.06, weak class (0.9×), rebuilder		2,188		0.38	4,500

Two behaviours worth noting: picks are worth **~17% more to a rebuilder than a contender** (discount + flexibility premium), and a future pick in a weak class one year out loses ~16% versus this year's equivalent. The `PICK_OUTCOMES` table in the engine is the **most placeholder-ish part of the whole model** — replace it with distributions built from your own historical value database as soon as you have two years of snapshots.

10.8 Roster-level output

Running all 13 archetypes as a single roster:

```
projected starter PPG        158.0
contender capital            77,200
rebuilder capital            52,449
now/future ratio             1.47
average age                  26.0
positional surplus           QB +1, RB 0, WR +1, TE 0, DL −1, LB −2, DB −1
recommended direction        CONTEND
```

The roster is old-skewing in value terms despite an average age of 26 — the contender/rebuilder capital ratio is the signal, not the average age. And the surplus row immediately tells this manager to spend their WR and QB depth on linebackers, which is the concrete, actionable output a roster analyzer should produce.

Part 11 — User-facing outputs

11.1 Player card

```
┌──────────────────────────────────────────────────────────────┐
│  PLAYER NAME              WR · 23.4 yrs · Year 2 · [team]    │
│                                                              │
│  DYNASTY   9,800   ▲ +340 (30d)        MARKET  7,900   ▼ −120 │
│  ┌──────────────┬──────────────┬──────────────┐              │
│  │ CONTENDER    │ BALANCED     │ REBUILDER    │              │
│  │    9,693     │    9,800     │    9,800     │              │
│  └──────────────┴──────────────┴──────────────┘              │
│  Rest of season: WR7      Model vs market: +1,900  ▲ STRONG BUY │
│  Realizable alpha: +1,463 (market dispersion HIGH — 4 of 10  │
│  sources rank him outside their top 20)                      │
│                                                              │
│  Floor (P20) 6,900 ·  Median 9,800 ·  Ceiling (P85) 12,400*  │
│  Confidence  ●●●○○      Volatility  ●●●●○                    │
│                                                              │
│  ┌── Expected career value ──────────────────────────┐       │
│  │          ┌────┐                                   │       │
│  │       ┌──┘    └──┐                                │       │
│  │     ┌─┘          └──┐                             │       │
│  │   ┌─┘               └────┐                        │       │
│  │ ┌─┘                      └─────┐                  │       │
│  │ 26  27  28  29  30  31  32  33   (shaded 80% band) │       │
│  └──────────────────────────────────────────────────┘       │
│                                                              │
│  WHY HE'S WORTH THIS                                         │
│   43%  production in year 3 and beyond                       │
│   40%  production in years 1–2                               │
│   17%  this season                                           │
│   +52% projected trajectory over 5 years (age curve + role growth) │
│   −21% survival drag                                         │
│                                                              │
│  P(above replacement, 2026)     88%                          │
│  P(top-12 WR season, any yr)    33%                          │
│  P(still a starter in 2029)     73%                          │
│  P(breakout next season)        45%                          │
│  P(loses most value in 1 yr)    16%                          │
│                                                              │
│  ⚠  Role: 68% route share — target share does not yet support the  │
```

```
|    ascension the model is pricing in. Watch weeks 1–4.            |
└──────────────────────────────────────────────────────────────────┘
```

* Ceiling shown on the value scale, derived from the P85 path, not from a single-season projection.

11.2 Plain-English explanation generation

Generate from templates keyed to the largest decomposition terms, so explanations are always consistent with the math:

> **Contender view.** "He ranks LB4 for contenders because he projects as a top-eight scorer at a position where he'll play every snap, he holds the green dot, and 31% of his value is available this season. His rebuild value is lower: his age-adjusted probability of still being a startable linebacker in 2029 is 66%, and linebacker value decays faster than edge value once snap share slips."

> **Market disagreement.** "The market prices him as the WR9 overall. Our model has him WR4. The difference is almost entirely role growth we're projecting and the market hasn't paid for yet — his 68% route share is the weak link. If route share reaches 80% by week 4, the model's number is right; if it doesn't, the market's is."

> **Sell flag.** "His current projection supports a top-15 season, but 85% of his total value sits in 2026. Effective age 30.9 after mileage adjustment; 1% probability of being startable in 2029. If you are not contending this year, he is a sell at any price above ~1,400."

[Recommendation] Cap explanations at three sentences and always name **the one variable that would change the conclusion**. That last clause is what makes the tool feel like an analyst rather than a calculator.

11.3 Buy / hold / sell logic

```
signal = STRONG BUY  if alpha > +900  and gap persisted ≥14d and liquidity > 0.5
         BUY         if alpha > +400
         HOLD        if |alpha| ≤ 400
         SELL        if alpha < −400
         STRONG SELL if alpha < −900  or (P(collapse_1y) > 0.5 and market > model)
```

Always display **why**: "sell because market > model" is a different recommendation from "sell because the collapse probability is 65%," and users need to know which one they're acting on.

11.4 Additional applications

Beyond the list you gave, this model architecture unlocks:

1. **League-market maker.** Compute every manager's implied strategy profile from their roster, then price each of your assets *in their currency*. This finds who overpays for what.
2. **Trade-timing calendar.** Value is horizon-dependent, so a player's contender value peaks around the trade deadline and his rebuilder value peaks in the offseason. Surface "sell this in October, buy it back in February."
3. **Portfolio risk view.** Sum σ across the roster with an assumed correlation structure (same-team players and same-position-group players are correlated) to show concentration risk. A roster of six high-σ young WRs is not diversified.
4. **Roster window forecast.** Project total starter PPG forward 4 seasons with the aging engine to show when the roster peaks — the single most persuasive output for a contend/rebuild decision.
5. **Scoring-settings simulator.** Because scoring is config, you can show "your roster in a tackle-heavy IDP league vs a big-play IDP league" — enormously useful for league commissioners setting rules.
6. **Startup draft board with pick-by-pick VONA.** Replacement level updates as the draft proceeds.
7. **Orphan-team evaluator.** Value an abandoned roster and give it a direction in one click.
8. **Auction dollar values.** `$ = budget × TV_i / Σ TV` over draftable players, with a stud-premium exponent.
9. **"What has to be true" tool.** Invert the model: given a player's market price, solve for the route share / snap share / target share that would justify it. This is the most differentiated feature on this list and falls straight out of having a parametric model.
10. **Injury-replacement finder.** When a starter goes down, rank waiver adds by *marginal* value to that specific lineup, not by absolute value.

11.5 Personalized roster analysis

Value is contextual. Apply, in this order:

```
1. Timeline:    evaluate with the manager's own strategy profile (inferred, overridable)
2. Need:        Need_g = 1 + 0.12·(starters_g − startable_g)/starters_g,  clip [0.85, 1.25]
```

```
3. Depth:       if startable_g > starters_g + 1, apply a consolidation discount to
                marginal assets at g (they can't all start)
4. Roster spot: charge C_spot per player beyond free spots
5. Risk budget: if roster σ concentration > 75th percentile, discount additional
                high-σ assets by 5%
```

Worked implications you asked about:

- *Contender with elite RBs upgrading a weak LB* — the LB upgrade is multiplied by `Need_LB` (up to 1.25) while the marginal RB is discounted by step 3. A trade that looks flat on raw values becomes clearly positive.
- *Rebuilder offered aging production for a young elite WR* — the rebuilder's `u(0) = 0.45` already crushes the veterans' value in his currency; the model will show the trade as heavily negative even at equal market value.
- *Team with excess depth paying a consolidation premium* — steps 3 and 4 combine to make `θ`-adjusted packaging favourable for them specifically.
- *Shallow team preferring quantity* — with many open roster spots, `C_spot` overflow is zero and their `Need` multipliers are high across several groups, so the same trade evaluates positively for them.

Roster metrics to compute: current-season projected starter PPG; contender capital; rebuilder capital; now/future ratio; competitive window (year of projected peak starter PPG); positional surplus/deficit; age concentration (share of value held by players 28+); risk concentration (share of value in top-quartile σ); bench value above replacement; future pick capital; roster liquidity (share of value in assets with high market dispersion — high liquidity means you can actually restructure).

Part 12 — Development roadmap

Phase 1 — Projection baseline + replacement engine (2–3 weeks)

Build. Scoring engine, projection ingest + blending, replacement engine with flex allocation, league config system, static JSON export. Output: VOR-based rankings for the upcoming season in any league format. **Data.** Projection sources (any 2+), your league configs, nflverse player IDs. **Benefit.** Immediately better than raw projection rankings, and it is the foundation every later phase sits on. **Risks.** Replacement buffers are the whole ballgame; wrong buffers produce plausible-looking but wrong cross-positional values. **Validation.** Reproduce known VBD results by hand for 5 players per position. Confirm Superflex raises QB replacement and TEP raises TE replacement. Confirm monotonicity in league size. **Dependencies.** None.

Phase 2 — Aging, survival, discounting (2–3 weeks)

Build. Aging curves, mileage, ascension `κ`, hazard model, horizon aggregation, the three strategy profiles, option-value season formula, probability outputs, explainability decomposition. **This is the phase that turns a redraft tool into a dynasty tool. Data.** DOB, draft data, career loads, snap history. **Benefit.** Contender/balanced/rebuilder values, career-value charts, buy/sell flags on fundamentals. **Risks.** Double-counting age via both the curve and the hazard; over-steep RB curves. **Validation.** `S(3)` calibration against your own historical retention rates. Ablate the option formula vs `μ−R`. Sanity-check that no 22-year-old with a 6-FPG projection outranks a 26-year-old star. **Dependencies.** Phase 1.

Phase 3 — IDP position models (2–3 weeks)

Build. True-position mapping and configurable groups (DL/EDGE/DT, DB/CB/S), IDP-specific curves and hazards, expected-sack and tackles-vs-expected regression, safety/CB archetypes, designation-risk scenarios. **Data.** Defensive snap counts and alignment; PFF/SIS if affordable, PBP-derived proxies if not. **Benefit.** This is your differentiator. Almost no public dynasty tool treats IDP as a first-class citizen, and your existing IDP anchor system is a workaround you can now replace with a real model. **Risks.** Charting data for alignment is the hardest dependency; without it, safety and CB archetypes are guesses. **Validation.** Per-position Spearman vs market; verify the LB/S scarcity behaviour reproduces; verify DT values swing appropriately between combined-DL and DT-required configs. **Dependencies.** Phases 1–2.

Phase 4 — Market comparison layer (1–2 weeks)

Build. Market ingest with type tags, dispersion, liquidity, gap, alpha, blended value, momentum guard, buy/sell/hold signals, historical value tracking. **Data.** Your 10-source scraper — **plus daily snapshots starting immediately. Benefit.** Buy-low/sell-high finder, model-vs-market pages, and the credibility of showing both numbers. **Risks.** Circularity — if you weight market too heavily, you have rebuilt KTC. Enforce that the fundamental value is computed and stored *before* any market data is read. **Validation.** Backtest T6: does a positive gap predict market appreciation over the following 6–12 months? **Dependencies.** Phases 1–2, plus ~6 months of snapshots before T6 is testable.

Phase 5 — Contract, news and role adjustments (2 weeks)

Build. Contract ingest from OverTheCap via nflverse, structured news-event system with decay and confidence, depth-chart competition detection, coordinator-change flags. **Data.** `load_contracts`, `load_depth_charts`, transaction feed. **Benefit.** Values respond to real events within hours instead

of waiting for projection updates. **Risks.** Event spam and manual-override creep. Keep the event enum closed and require confidence scores. **Validation.** Replay 20 known 2024–25 events (a trade, a coordinator change, a draft pick at the same position) and confirm the value moves are directionally right and proportionate. **Dependencies.** Phase 2.

Phase 6 — Roster-level strategy (2 weeks)

Build. Sleeper league sync, roster valuation, timeline inference, need multipliers, window forecast, positional surplus, risk concentration, contend/retool/rebuild recommendation. **Data.** Sleeper API. **Benefit.** Personalization — the thing that turns a rankings site into a tool people return to weekly. **Risks.** Timeline inference will be wrong for some rosters; always let the user override. **Validation.** Run it on 20 real rosters and check the recommended direction against what an experienced manager would say. **Dependencies.** Phases 1–4.

Phase 7 — Trade finder and optimization (2–3 weeks)

Build. CES package math, roster-spot costs, need multipliers, **double-positive** trade search across league rosters, trade-block recommendations, 2-for-1 and 3-for-2 search with pruning. **Data.** All league rosters from Sleeper sync. **Benefit.** The highest-engagement feature in this entire document. "Here are 6 trades your leaguemates should accept" is a product, not a feature. **Risks.** Combinatorial explosion — prune by value band and position need before scoring. **Validation.** Measure acceptance rate of suggested trades among your users. That is a real, direct KPI. **Dependencies.** Phases 1–6.

Phase 8 — ML refinement (ongoing, offseason)

Build. Three targeted sub-models: next-season role change (κ), games missed, and starter-loss hazard. Each replaces a parametric component while keeping the skeleton. **Data.** 8+ years of clean as-of snapshots. **Benefit.** Incremental accuracy where the parametric priors are weakest. **Risks.** Explainability regression; drift. Require each sub-model to beat its parametric counterpart on two of three folds *and* ship with SHAP-based card explanations. **Validation.** Same rolling-origin protocol; ablate against the parametric version. **Dependencies.** Everything, plus historical data you must start collecting now.

Part 13 — Open questions and limitations

What cannot be resolved without proprietary data

1. **Alignment and coverage-role data for IDP.** Box/slot/deep snap splits, pass-rush snaps, pressure rate and targets-faced are PFF/SIS products. Public PBP proxies exist but are materially worse. Without them, safety archetypes are inferred from tackle rate, and CB modelling is genuinely weak. **This is the largest single gap in the model.**
2. **Expected sacks and tackles vs expected.** Both are proprietary constructions. You can approximate expected sacks from pass-rush snaps × a rolling pressure proxy, but the approximation is coarse.
3. **True projection-source accuracy history.** Without archived preseason projections you cannot compute honest source weights for past seasons. Start archiving now; accept equal weights for year 1.
4. **Real dynasty trade flow.** FantasyCalc sees actual trades; you see published values. Your liquidity measure is therefore a proxy built from cross-source dispersion, not observed volume.

What will remain uncertain no matter what

5. **Rookie valuation before an NFL snap.** The κ term is doing enormous work with weak information. Rookie values will always have the widest bands, and that is correct rather than a flaw.
6. **Coaching and scheme effects.** No public quantification is reliable enough to model as a continuous variable. Binary stability flags are the honest ceiling.
7. **The discount rate.** There is no observable "market rate" for future fantasy points. d is a preference parameter you can calibrate to market prices, but calibrating it to the market and then claiming to disagree with the market is partly circular. Be transparent that d is a choice.
8. **Regime change.** The NFL's usage patterns shift — RB committee rates, tight-end usage, defensive alignment trends. Curves fitted on 2010–2020 may mis-fit 2027. Refit annually with recency weighting.
9. **Small-sample IDP positions.** DT and CB have few fantasy-relevant seasons per year. Hierarchical pooling helps; it does not fix it.

Where human judgment stays necessary

10. **Designation calls.** Whether a hybrid defender will be listed as LB or DL next season is a judgment, not a computation. Model it as a probability and let a human set it.
11. **News interpretation.** Deciding whether a beat report is confidence 0.4 or 0.8 is editorial. The system should structure that judgment, not eliminate it.

12. **Archetype classification at the margins.** Continuous blend weights help, but someone has to decide what a `w_dual = 0.5` quarterback is.

Honest statement of what this model is not

It is not validated. Every parameter in Part 6 is a prior chosen from published research and reasoning about mechanism. The architecture is defensible; the numbers are hypotheses. **Do not publish confidence intervals from this model to users until the calibration work in Part 8 is done** — showing an uncalibrated "73% chance he's still a starter" is worse than showing nothing.

Part 14 — Final recommendation

Build this, in this order, starting now.

The Version 1 formula — commit to exactly this:

```
For each player i, for each strategy s ∈ {contender, balanced, rebuilder}:

  μ(0)  = weighted trimmed mean of projection sources, scored under league config
  R_p   = FPG at rank [ league-wide startable slots (flex-allocated) + waiver buffer ]
  μ(t)  = μ(0) · A_p(a_eff(t)) / A_p(a_eff(0)) · [1 + κ(1 − e^(−t/1.5))]
  σ(t)  = sqrt( (CV_p·(1+0.9·role_unc)(1+0.45·event_vol)(1+0.6·small_sample)·μ(t))²
                + σ_src² + (ψ_p·μ(t)·√t)² )
  SSV(t)= [ (μ(t) − R_p)·Φ(z) + σ(t)·φ(z) ] · G(t),        z = (μ(t)−R_p)/σ(t)
  S(t)  = Π (1 − h_p(age, role, contract, durability, draft capital × decay))

  DV_s  = Σ_t u_s(t) · d_s^t · S(t) · SSV(t)  −  λ_s · 0.35 · Ψ
  TV_s  = 10000 · (DV_s / anchor_s)^1.20

  Market layer computed strictly afterward:  Gap = TV_balanced − TV_market
                                             Alpha = Gap × Liquidity
  Packages: ( Σ v^1.20 )^(1/1.20) − 120 · roster-spot overflow
```

Architecture: transparent parametric, nine modules, PostgreSQL with `as_of` snapshots, Python engine, FastAPI + static JSON export, everything versioned by `(model_version, param_set_id, config_id, as_of)`.

The five decisions I'd defend hardest:

1. **`E[max(0, μ−R)]`, not `μ−R`.** Uncertainty becomes a modelled quantity rather than a fudge factor, and upside gets priced without a youth bonus.
2. **Aging and survival kept numerically separate.** Production-if-playing and probability-of-playing are different questions, and merging them is the most common failure mode in public dynasty models.
3. **Replacement level does all the positional-scarcity work.** No hand-set position multipliers anywhere — not for Superflex, not for TE premium, not for IDP. Format sensitivity is a property of the config, not of a lookup table.
4. **Three currencies, not one number with a multiplier.** Strategy enters through `u_s(t)`, `d_s` and `λ_s` inside the sum. That's what makes the contender/rebuilder distinction mean something, and it's what enables double-positive trade search.
5. **The market layer runs last and is stored separately.** Fundamental value is computed with no knowledge of market value. That is the only way the model can honestly disagree.

The three things to do this week, before writing any model code:

1. **Start daily market snapshots** with `as_of` timestamps from your existing 10-source scraper. You cannot backtest market alpha without this, and every day you wait is a day of data you never get back.
2. **Start weekly nflverse snapshots** — rosters, snap counts, depth charts, injuries, contracts.
3. **Build the league config + scoring engine + replacement engine** (Phase 1). It's two weeks, it's useful on its own, and it's the piece everything else depends on.

The first thing to test once you have projections: the option-value ablation in §8.6. If `E[max(0, μ−R)]` does not beat `μ−R` on three-year realised VOR, my central design claim is wrong and you should simplify to the plain difference. I'd rather you find that out in week 6 than believe it for three years.

What "done" looks like for v1: a values table with contender/balanced/rebuilder trade values and a market gap for every rostered offensive and defensive player in your league format, a player card that explains any of those numbers in three sentences, and a logged backtest showing the fundamental model beats a KTC/FantasyCalc blend on three-year realised value. That is a genuinely differentiated product, and none of it requires machine learning.

Source list

Aging and career: 4for4 production curves · PFF aging curves by league year · PFF metrics that matter: aging · ESPN peak/decline · FantasyPros RB decline · Apex RB peak age · Apex QB peak age / archetypes · FTN over-30 study · Footballguys — Harstad on aging-curve bias · Advanced Football Analytics — QB aging & delta method · Football Perspective — QB age curves · NFL career length by position

Predictive metrics: Sharp Football — WR stats that matter · SumerSports — YPRR stability · SumerSports — sticky stats · PFF — YPRR · Fantasy Life — TE metrics 2026 · Fantasy Points — YPRR defense

IDP: Yahoo/Athlon/Lindy's IDP advanced stats · Fantasy Life — IDP stats that move the needle · Fantasy Life — evaluating defense · The IDP Show — production vs expected · The IDP Show — LB tackle efficiency · PFF — IDP tackles and sacks · PFF — dynasty IDP true position · NFL.com — IDP strategy · Fantasy Six Pack — IDP age/peak value · DraftSharks dynasty IDP methodology

Draft capital, college, rookies: Fantasy Footballers — draft capital and early production · Dynasty Nerds — draft capital · Roster survival by draft slot · PlayerProfiler — breakout age origin · PFF — dominator + breakout age · Apex — rookie WR breakout profile · NBC — rookie pick hit rates · Dynasty Nerds — pick hit rates by slot

Injury: Systematic review — previous injury as risk factor · Prospective elite-football injury study · Combined risk factors / injury-prone hazard · Draft Sharks injury model inputs · Fantasy Points — counterpoint

Valuation frameworks and market: FantasyPros VBD/VORP primer · FantasyPros VBD 2026 · Draft Day Authority VBD reference · KeepTradeCut · DynastyProcess calculator · FantasyPros dynasty tool comparison

Projections and data: FFA projection accuracy · FFA DFS accuracy + weighting method · FFA positional bias · nflreadr function reference · nflreadpy · nflverse snap counts / PFR

Appendices — complete source

Appendix A — `dynasty_engine.py`

The complete valuation engine. Stdlib only — no numpy, no pandas — so it runs anywhere, including a cron job on the VPS. Every function referenced in Part 5 is here under the name given there.

```
"""
dynasty_engine.py — Reference implementation of the Brisket Dynasty Valuation Model (v1.0)

Design goals:
  * Glass-box: every number is traceable to a named parameter.
  * Projection-source agnostic: swap the projection file, keep the model.
  * League-format agnostic: scoring, lineup slots and replacement levels are config.
  * Offense and IDP are the same code path; only parameters differ.

No third-party dependencies (stdlib only) so it can run anywhere, including the VPS.

ALL PARAMETERS BELOW ARE STARTING PRIORS. They are informed by published research
but are NOT backtested truth. Treat params.py as the thing you tune, not the code.
"""

from __future__ import annotations
import math
import json
from dataclasses import dataclass, field, asdict
from typing import Dict, List, Optional, Tuple

SQRT2 = math.sqrt(2.0)
SQRT2PI = math.sqrt(2.0 * math.pi)


def _phi(z: float) -> float:
    """Standard normal PDF."""
    return math.exp(-0.5 * z * z) / SQRT2PI


def _Phi(z: float) -> float:
    """Standard normal CDF."""
    return 0.5 * (1.0 + math.erf(z / SQRT2))


def _clamp(x: float, lo: float, hi: float) -> float:
```

```
    return max(lo, min(hi, x))


# ---------------------------------------------------------------------------
# 1. LEAGUE CONFIGURATION
# ---------------------------------------------------------------------------

POSITIONS = ["QB", "RB", "WR", "TE", "DT", "EDGE", "LB", "CB", "S"]

# Fantasy-slot groups. Real position -> lineup group. Configurable for leagues
# that collapse DT/EDGE into "DL" or CB/S into "DB".
DEFAULT_POS_GROUPS = {
    "QB": "QB", "RB": "RB", "WR": "WR", "TE": "TE",
    "DT": "DL", "EDGE": "DL", "LB": "LB", "CB": "DB", "S": "DB",
}


@dataclass
class ScoringConfig:
    # offense
    pass_yd: float = 0.04
    pass_td: float = 4.0
    interception: float = -2.0
    rush_yd: float = 0.1
    rush_td: float = 6.0
    rec: float = 1.0                # PPR
    rec_te_bonus: float = 0.5       # TE premium (additional per reception)
    rec_yd: float = 0.1
    rec_td: float = 6.0
    fumble_lost: float = -2.0
    first_down: float = 0.0
    # IDP
    tackle_solo: float = 1.5
    tackle_assist: float = 0.75
    sack: float = 4.0
    tfl: float = 1.0
    qb_hit: float = 0.5
    pass_defended: float = 1.5
    interception_def: float = 4.0
    forced_fumble: float = 3.0
    fumble_recovery: float = 2.0
    def_td: float = 6.0
    safety: float = 2.0


@dataclass
class LeagueConfig:
    teams: int = 12
    # starting lineup: fixed slots by group
    starters: Dict[str, int] = field(default_factory=lambda: {
        "QB": 1, "RB": 2, "WR": 3, "TE": 1, "DL": 2, "LB": 3, "DB": 3
    })
    # flex slots: name -> (count_per_team, eligible groups)
    flex: Dict[str, Tuple[int, List[str]]] = field(default_factory=lambda: {
        "FLEX": (1, ["RB", "WR", "TE"]),
        "SUPERFLEX": (1, ["QB", "RB", "WR", "TE"]),
        "IDP_FLEX": (1, ["DL", "LB", "DB"]),
    })
    # waiver buffer: how many extra players per team beyond starters are rostered
    # AND startable-quality at that group. Drives replacement level.
    waiver_buffer: Dict[str, float] = field(default_factory=lambda: {
        "QB": 0.85, "RB": 0.75, "WR": 1.00, "TE": 0.40,
        "DL": 0.50, "LB": 0.50, "DB": 0.50
    })
    roster_size: int = 35
    taxi_size: int = 5
    best_ball: bool = False
    pos_groups: Dict[str, str] = field(default_factory=lambda: dict(DEFAULT_POS_GROUPS))
    scoring: ScoringConfig = field(default_factory=ScoringConfig)

    def group(self, pos: str) -> str:
        return self.pos_groups[pos]
```

```
    @property
    def superflex(self) -> bool:
        return any("QB" in elig for _, elig in self.flex.values())


# ---------------------------------------------------------------------------
# 2. SCORING ENGINE — raw projected stats -> fantasy points
# ---------------------------------------------------------------------------

def score_stats(stats: Dict[str, float], pos: str, sc: ScoringConfig) -> float:
    """Convert a raw projected stat line into fantasy points for this league."""
    p = 0.0
    p += stats.get("pass_yd", 0) * sc.pass_yd
    p += stats.get("pass_td", 0) * sc.pass_td
    p += stats.get("int", 0) * sc.interception
    p += stats.get("rush_yd", 0) * sc.rush_yd
    p += stats.get("rush_td", 0) * sc.rush_td
    rec = stats.get("rec", 0)
    p += rec * sc.rec
    if pos == "TE":
        p += rec * sc.rec_te_bonus
    p += stats.get("rec_yd", 0) * sc.rec_yd
    p += stats.get("rec_td", 0) * sc.rec_td
    p += stats.get("fum_lost", 0) * sc.fumble_lost
    p += stats.get("first_down", 0) * sc.first_down
    # IDP
    p += stats.get("tkl_solo", 0) * sc.tackle_solo
    p += stats.get("tkl_ast", 0) * sc.tackle_assist
    p += stats.get("sack", 0) * sc.sack
    p += stats.get("tfl", 0) * sc.tfl
    p += stats.get("qb_hit", 0) * sc.qb_hit
    p += stats.get("pd", 0) * sc.pass_defended
    p += stats.get("int_def", 0) * sc.interception_def
    p += stats.get("ff", 0) * sc.forced_fumble
    p += stats.get("fr", 0) * sc.fumble_recovery
    p += stats.get("def_td", 0) * sc.def_td
    p += stats.get("safety", 0) * sc.safety
    return p


# ---------------------------------------------------------------------------
# 3. PROJECTION BLENDING
# ---------------------------------------------------------------------------

@dataclass
class ProjectionSource:
    name: str
    fpg: float                    # projected fantasy points per game (league scoring)
    games: float = 16.0
    weight: float = 1.0           # from historical MAE-based accuracy, per position


def blend_projections(sources: List[ProjectionSource],
                      trim_frac: float = 0.10) -> Tuple[float, float, float]:
    """
    Weighted trimmed mean of per-game projections.
    Returns (mu_fpg, sigma_source, games).

    sigma_source = weighted SD across sources = the *disagreement* component of
    projection uncertainty. It is NOT the full outcome distribution.
    """
    if not sources:
        raise ValueError("no projection sources")
    if len(sources) >= 5 and trim_frac > 0:
        s = sorted(sources, key=lambda x: x.fpg)
        k = max(1, int(len(s) * trim_frac))
        s = s[k:-k] if len(s) - 2 * k >= 3 else s
    else:
        s = list(sources)
    W = sum(x.weight for x in s)
    mu = sum(x.fpg * x.weight for x in s) / W
    games = sum(x.games * x.weight for x in s) / W
    var = sum(x.weight * (x.fpg - mu) ** 2 for x in s) / W
```

```
    return mu, math.sqrt(var), games


# ---------------------------------------------------------------------------
# 4. REPLACEMENT-LEVEL ENGINE
# ---------------------------------------------------------------------------

class ReplacementEngine:
    """
    Dynamic replacement level driven by lineup requirements, flex eligibility,
    league size and waiver depth.

    pool: group -> callable(rank:int) -> projected FPG at that positional rank.
    In production this is the sorted projection list. Here it can also be a
    synthetic decay curve for demos.
    """

    def __init__(self, cfg: LeagueConfig, pool: Dict[str, callable]):
        self.cfg = cfg
        self.pool = pool
        self.counts: Dict[str, int] = {}
        self.replacement: Dict[str, float] = {}
        self._solve()

    def _solve(self):
        cfg = self.cfg
        groups = list(cfg.starters.keys())
        counts = {g: cfg.teams * cfg.starters.get(g, 0) for g in groups}

        # Greedy flex allocation: each flex slot goes to whichever eligible group
        # currently offers the best next-available player.
        for _slot, (n_per_team, elig) in cfg.flex.items():
            for _ in range(n_per_team * cfg.teams):
                best_g, best_v = None, -1e9
                for g in elig:
                    if g not in self.pool:
                        continue
                    v = self.pool[g](counts.get(g, 0) + 1)
                    if v > best_v:
                        best_g, best_v = g, v
                if best_g:
                    counts[best_g] = counts.get(best_g, 0) + 1

        # Waiver buffer -> the true replacement rank
        for g in counts:
            buf = int(round(cfg.teams * cfg.waiver_buffer.get(g, 0.5)))
            rank = counts[g] + buf
            self.counts[g] = counts[g]
            self.replacement[g] = self.pool[g](rank) if g in self.pool else 0.0

    def R(self, group: str) -> float:
        return self.replacement.get(group, 0.0)


def synthetic_pool(top_fpg: float, decay: float):
    """Exponential rank-decay curve used for demos until real projections land."""
    return lambda rank: top_fpg * math.exp(-decay * max(0, rank - 1))


# ---------------------------------------------------------------------------
# 5. AGING / TRAJECTORY ENGINE
# ---------------------------------------------------------------------------

# (peak_age, curvature_before_peak, curvature_after_peak)
AGE_CURVES: Dict[str, Tuple[float, float, float]] = {
    "QB_pocket": (30.0, 0.006, 0.010),
    "QB_dual":   (26.0, 0.010, 0.028),
    "RB":        (25.0, 0.012, 0.030),
    "WR":        (27.0, 0.014, 0.024),
    "TE":        (27.5, 0.030, 0.018),
    "DT":        (27.5, 0.016, 0.020),
    "EDGE":      (27.0, 0.014, 0.022),
    "LB":        (26.5, 0.012, 0.030),
```

```
    "CB":        (26.0, 0.010, 0.030),
    "S":         (27.0, 0.012, 0.026),
}

# career-load mileage: (reference_load, load_per_effective_year, cap_years)
MILEAGE = {
    "RB":   ("touches", 750.0, 500.0, 2.5),
    "WR":   ("targets", 500.0, 450.0, 1.5),
    "TE":   ("targets", 400.0, 450.0, 1.5),
    "QB":   ("dropbacks", 2000.0, 2500.0, 1.0),
    "LB":   ("snaps", 3000.0, 2200.0, 1.5),
    "EDGE": ("snaps", 3000.0, 2400.0, 1.5),
    "DT":   ("snaps", 3000.0, 2400.0, 1.0),
    "CB":   ("snaps", 3000.0, 2400.0, 1.5),
    "S":    ("snaps", 3000.0, 2400.0, 1.5),
}


def curve_key(pos: str, archetype: Optional[str]) -> str:
    if pos == "QB":
        return "QB_dual" if archetype == "dual" else "QB_pocket"
    return pos


def aging_mult(pos: str, age: float, archetype: Optional[str] = None) -> float:
    """Multiplicative production factor, conditional on the player still playing."""
    peak, c_up, c_dn = AGE_CURVES[curve_key(pos, archetype)]
    if age <= peak:
        return math.exp(-c_up * (peak - age) ** 2)
    return math.exp(-c_dn * (age - peak) ** 2)


def effective_age(pos: str, age: float, career_load: float) -> float:
    """Chronological age + mileage penalty (applied only on the decline side)."""
    if pos not in MILEAGE:
        return age
    _unit, ref, per_year, cap = MILEAGE[pos]
    extra = _clamp((career_load - ref) / per_year, 0.0, cap)
    return age + extra


def ascension_mult(kappa: float, t: int, tau: float = 1.5) -> float:
    """Role growth for ascending young players. Saturating, bounded by kappa."""
    if kappa <= 0 or t <= 0:
        return 1.0
    return 1.0 + kappa * (1.0 - math.exp(-t / tau))


# ---------------------------------------------------------------------------
# 6. SURVIVAL / LONGEVITY ENGINE
# ---------------------------------------------------------------------------

# Base annual hazard of losing fantasy-startable status, by position and age band.
BASE_HAZARD = {
    "QB":   [(24, 0.16), (27, 0.10), (30, 0.09), (33, 0.14), (36, 0.24), (99, 0.38)],
    "RB":   [(24, 0.11), (26, 0.15), (27, 0.22), (28, 0.30), (29, 0.38), (99, 0.50)],
    "WR":   [(24, 0.13), (27, 0.09), (29, 0.13), (31, 0.22), (33, 0.34), (99, 0.46)],
    "TE":   [(24, 0.16), (27, 0.10), (30, 0.12), (32, 0.20), (34, 0.32), (99, 0.44)],
    "DT":   [(24, 0.14), (27, 0.10), (30, 0.14), (32, 0.24), (99, 0.38)],
    "EDGE": [(24, 0.14), (27, 0.10), (30, 0.15), (32, 0.26), (99, 0.40)],
    "LB":   [(24, 0.15), (27, 0.12), (29, 0.18), (31, 0.28), (99, 0.42)],
    "CB":   [(24, 0.18), (27, 0.15), (29, 0.22), (31, 0.32), (99, 0.46)],
    "S":    [(24, 0.16), (27, 0.13), (29, 0.19), (31, 0.29), (99, 0.44)],
}


def base_hazard(pos: str, age: float) -> float:
    for cutoff, h in BASE_HAZARD[pos]:
        if age < cutoff:
            return h
    return 0.5
```

```
@dataclass
class RiskProfile:
    """All 0-1 scores. Higher = better/safer unless noted."""
    role_security: float = 0.6      # snap/route/target-share lock on the job
    draft_capital: float = 0.5      # 1.0 = top-10 pick, 0.0 = UDFA
    contract_security: float = 0.6  # years remaining x guarantees
    durability: float = 0.6         # inverse of injury burden
    production_z: float = 0.0       # z-score of current VOR vs position
    scheme_stability: float = 0.6
    role_uncertainty: float = 0.25  # 0 = role fully known, 1 = total unknown (RISK: higher=worse)
    event_volatility: float = 0.30  # dependence on TDs/sacks/INTs (RISK: higher=worse)
    small_sample: float = 0.0       # 1.0 = no NFL sample at all (RISK: higher=worse)
    designation_risk: float = 0.0   # IDP position-eligibility change risk (higher=worse)


def hazard(pos: str, age: float, rp: RiskProfile, dc_decay: float) -> float:
    """Annual hazard, base rate adjusted by role/contract/durability/production."""
    h = base_hazard(pos, age)
    adj = 1.0
    adj *= (1.0 + 0.55 * (0.6 - rp.role_security))
    adj *= (1.0 + 0.30 * dc_decay * (0.5 - rp.draft_capital))
    adj *= (1.0 + 0.25 * (0.6 - rp.contract_security))
    adj *= (1.0 + 0.35 * (0.6 - rp.durability))
    adj *= (1.0 - 0.18 * _clamp(rp.production_z, -2.0, 2.0))
    adj *= (1.0 + 0.20 * rp.designation_risk)
    return _clamp(h * adj, 0.02, 0.85)


# ---------------------------------------------------------------------------
# 7. UNCERTAINTY MODEL
# ---------------------------------------------------------------------------

# Baseline coefficient of variation of a player's SEASONAL PPG outcome
# (not week-to-week noise), and per-year drift volatility for future seasons.
CV_BASE = {"QB": 0.26, "RB": 0.36, "WR": 0.34, "TE": 0.36,
           "DT": 0.32, "EDGE": 0.34, "LB": 0.26, "CB": 0.40, "S": 0.30}
DRIFT_VOL = {"QB": 0.16, "RB": 0.30, "WR": 0.22, "TE": 0.22,
             "DT": 0.20, "EDGE": 0.22, "LB": 0.22, "CB": 0.30, "S": 0.24}


def sigma_fpg(pos: str, mu: float, rp: RiskProfile, sigma_source: float, t: int) -> float:
    """Dispersion of a player's realised seasonal PPG, t years from now."""
    cv = CV_BASE[pos]
    cv *= (1.0 + 0.90 * rp.role_uncertainty)
    cv *= (1.0 + 0.45 * rp.event_volatility)
    cv *= (1.0 + 0.60 * rp.small_sample)
    s0 = math.sqrt((cv * mu) ** 2 + sigma_source ** 2)
    drift = DRIFT_VOL[pos] * mu * math.sqrt(max(0, t))
    return math.sqrt(s0 ** 2 + drift ** 2)


# ---------------------------------------------------------------------------
# 8. SEASON VALUE — expected starter surplus (option-value formulation)
# ---------------------------------------------------------------------------

def season_starter_value(mu: float, sigma: float, R: float, games: float) -> float:
    """
    E[ max(0, FPG - R) ] * games, under FPG ~ Normal(mu, sigma).

    Why max(0, .): you are never forced to start a below-replacement player, so
    downside is truncated. This is what gives an uncertain, low-projected young
    player real dynasty value without hand-tuned "upside bonuses", and what makes
    a 250-point projection with a narrow range differ from a wide one.
    """
    if sigma <= 1e-9:
        return max(0.0, mu - R) * games
    z = (mu - R) / sigma
    return ((mu - R) * _Phi(z) + sigma * _phi(z)) * games


# ---------------------------------------------------------------------------
# 9. STRATEGY PROFILES
# ---------------------------------------------------------------------------
```

```
@dataclass
class Strategy:
    name: str
    discount: float               # per-year discount factor
    horizon: int                  # seasons modelled beyond the current one
    usability: List[float]        # weight on each future season's points to THIS roster
    risk_lambda: float            # >0 risk averse, <0 risk seeking


STRATEGIES = {
    "contender": Strategy("contender", 0.72, 4, [1.00, 0.95, 0.80, 0.65, 0.55], 0.20),
    "balanced":  Strategy("balanced", 0.85, 6, [1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80], 0.10),
    "rebuilder": Strategy("rebuilder", 0.93, 8, [0.45, 0.75, 1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80], -0.10),
}


# ---------------------------------------------------------------------------
# 10. PLAYER + CORE VALUATION
# ---------------------------------------------------------------------------

@dataclass
class Player:
    player_id: str
    name: str
    pos: str
    age: float
    nfl_season: int                       # 1 = rookie year
    fpg: float                            # blended projected FPG, this season
    games: float = 16.0
    sigma_source: float = 0.0
    archetype: Optional[str] = None       # "dual"/"pocket" QB; "box"/"deep" S; "slot"/"boundary" CB
    career_load: float = 0.0              # touches / targets / snaps per MILEAGE
    kappa: float = 0.0                    # ascension coefficient (role growth ahead)
    risk: RiskProfile = field(default_factory=RiskProfile)
    market_value: Optional[float] = None  # 0-10000 external market value
    market_dispersion: float = 0.20       # expert/market disagreement, 0-1


@dataclass
class Valuation:
    player_id: str
    name: str
    pos: str
    raw: Dict[str, float]
    seasons: List[Dict[str, float]]
    fundamental: Dict[str, float]         # strategy -> raw discounted value
    trade_value: Dict[str, float]         # strategy -> 0-10000
    blended_value: Optional[float]
    market_gap: Optional[float]
    probs: Dict[str, float]
    explain: Dict[str, float]


class DynastyModel:
    def __init__(self, cfg: LeagueConfig, repl: ReplacementEngine,
                 anchors: Optional[Dict[str, float]] = None,
                 stud_gamma: float = 1.20,
                 pool: Optional[Dict[str, callable]] = None):
        self.cfg = cfg
        self.repl = repl
        # one anchor per strategy: values are comparable WITHIN a profile
        self.anchors = anchors or {k: 1.0 for k in STRATEGIES}
        self.stud_gamma = stud_gamma
        self.pool = pool or {}

    def calibrate(self, players: List["Player"], target: float = 9800.0) -> None:
        """Set anchors so the most valuable asset in the pool scores `target`."""
        maxima = {k: 1e-9 for k in STRATEGIES}
        for p in players:
            for k, v in self._fundamental_only(p).items():
                maxima[k] = max(maxima[k], v)
        self.anchors = {k: m / ((target / 10000.0) ** (1.0 / self.stud_gamma))
```

```
                    for k, m in maxima.items()}

def _fundamental_only(self, p: "Player") -> Dict[str, float]:
    out = {}
    for key, st in STRATEGIES.items():
        rows = self.season_path(p, st.horizon)
        v = 0.0
        for r in rows:
            t = int(r["t"])
            u = st.usability[min(t, len(st.usability) - 1)]
            v += u * (st.discount ** t) * r["expected"]
        disp = math.sqrt(sum(((st.discount ** int(r["t"])) * r["sigma"] * r["games"]
                              * r["survival"]) ** 2 for r in rows))
        out[key] = max(0.0, v - st.risk_lambda * 0.35 * disp)
    return out

# -- season path ------------------------------------------------------
def season_path(self, p: Player, horizon: int) -> List[Dict[str, float]]:
    g = self.cfg.group(p.pos)
    R = self.repl.R(g)
    a0_eff = effective_age(p.pos, p.age, p.career_load)
    A0 = aging_mult(p.pos, a0_eff, p.archetype)
    dc_decay = math.exp(-0.45 * max(0, p.nfl_season - 1))  # draft capital fades

    rows, S = [], 1.0
    for t in range(0, horizon + 1):
        age_t = p.age + t
        a_eff = effective_age(p.pos, age_t, p.career_load + t * self._load_rate(p))
        A = aging_mult(p.pos, a_eff, p.archetype)
        mu = p.fpg * (A / A0) * ascension_mult(p.kappa, t)
        sig = sigma_fpg(p.pos, max(mu, 0.1), p.risk, p.sigma_source, t)
        games = p.games if t == 0 else 17.0 * (0.86 + 0.10 * p.risk.durability)
        ssv = season_starter_value(mu, sig, R, games)
        if t > 0:
            S *= (1.0 - hazard(p.pos, age_t, p.risk, dc_decay))
        rows.append({"t": t, "age": age_t, "age_eff": a_eff, "mu": mu,
                     "sigma": sig, "games": games, "R": R,
                     "ssv": ssv, "survival": S, "expected": ssv * S})
    return rows

def _load_rate(self, p: Player) -> float:
    return {"RB": 260.0, "WR": 120.0, "TE": 90.0, "QB": 560.0}.get(p.pos, 700.0)

# -- aggregate --------------------------------------------------------
def value(self, p: Player) -> Valuation:
    fundamental, seasons_cache = {}, None
    for key, st in STRATEGIES.items():
        rows = self.season_path(p, st.horizon)
        if key == "balanced":
            seasons_cache = rows
        v = 0.0
        for r in rows:
            t = int(r["t"])
            u = st.usability[min(t, len(st.usability) - 1)]
            v += u * (st.discount ** t) * r["expected"]
        # certainty equivalent: penalise (or reward) dispersion of the path
        disp = math.sqrt(sum(((st.discount ** int(r["t"])) * r["sigma"] * r["games"]
                              * r["survival"]) ** 2 for r in rows))
        v_ce = max(0.0, v - st.risk_lambda * 0.35 * disp)
        fundamental[key] = v_ce

    rows = seasons_cache
    R = rows[0]["R"]
    probs = self._probabilities(p, rows, R)
    tv = {k: self.to_trade_value(v, k) for k, v in fundamental.items()}

    blended, gap = None, None
    if p.market_value is not None:
        blended = self.blend_market(p, tv["balanced"])
        gap = tv["balanced"] - p.market_value

    return Valuation(
        player_id=p.player_id, name=p.name, pos=p.pos,
```

```
        raw={"fpg": p.fpg, "replacement": R, "vorg": p.fpg - R,
             "age": p.age, "age_eff": rows[0]["age_eff"]},
        seasons=rows, fundamental=fundamental, trade_value=tv,
        blended_value=blended, market_gap=gap, probs=probs,
        explain=self.explain(p, rows, fundamental["balanced"]),
    )

# -- probability outputs ------------------------------------------------
def _probabilities(self, p: Player, rows: List[Dict[str, float]], R: float) -> Dict[str, float]:
    r0 = rows[0]
    z0 = (r0["mu"] - R) / r0["sigma"]
    p_above = _Phi(z0)
    # "elite" = the FPG of a top-(teams/2) finisher at this group in the pool
    g = self.cfg.group(p.pos)
    if g in self.pool:
        elite_line = self.pool[g](max(1, self.cfg.teams // 2))
    else:
        elite_line = R + 1.75 * max(1.0, 0.45 * R)
    p_elite = 1.0 - _Phi((elite_line - r0["mu"]) / r0["sigma"])
    # starter in 3 years
    idx3 = min(3, len(rows) - 1)
    r3 = rows[idx3]
    p_start3 = r3["survival"] * _Phi((r3["mu"] - R) / r3["sigma"])
    # value collapse within one year
    r1 = rows[min(1, len(rows) - 1)]
    p_collapse = 1.0 - r1["survival"] * _Phi((r1["mu"] - R) / r1["sigma"])
    # breakout: exceeds current projection by >35% next year
    p_break = 1.0 - _Phi((1.35 * r0["mu"] - r1["mu"]) / r1["sigma"])
    return {"p_above_replacement": p_above, "p_elite_season": p_elite,
            "p_starter_in_3y": p_start3, "p_bust_year1": 1.0 - p_above,
            "p_value_collapse_1y": p_collapse, "p_breakout": p_break}

# -- scaling ------------------------------------------------------------
def to_trade_value(self, v: float, strategy: str = "balanced") -> float:
    a = self.anchors.get(strategy, 1.0)
    return 10000.0 * ((max(0.0, v) / a) ** self.stud_gamma)

# -- market -------------------------------------------------------------
def blend_market(self, p: Player, fundamental_tv: float) -> float:
    """
    Bayesian-style blend. Market precision is HIGHER for established
    offensive veterans (deep, liquid market) and LOWER for IDP, rookies and
    deep-bench assets (thin, hype-driven market).
    """
    tau_market = 1.0
    if self.cfg.group(p.pos) in ("DL", "LB", "DB"):
        tau_market *= 0.45
    if p.nfl_season <= 1:
        tau_market *= 0.55
    tau_market *= (1.0 - 0.5 * p.market_dispersion)
    tau_model = 0.85 * (1.0 - 0.4 * p.risk.small_sample)
    w = tau_model / (tau_model + tau_market)
    return w * fundamental_tv + (1 - w) * p.market_value

# -- explainability -----------------------------------------------------
def explain(self, p: Player, rows: List[Dict[str, float]], total: float) -> Dict[str, float]:
    st = STRATEGIES["balanced"]
    contrib = []
    for r in rows:
        t = int(r["t"])
        u = st.usability[min(t, len(st.usability) - 1)]
        contrib.append(u * (st.discount ** t) * r["expected"])
    s = sum(contrib) or 1.0
    return {
        "year_0_share": contrib[0] / s,
        "years_1_2_share": sum(contrib[1:3]) / s,
        "years_3_plus_share": sum(contrib[3:]) / s,
        "survival_drag": 1.0 - (sum(r["expected"] for r in rows) /
                                max(1e-9, sum(r["ssv"] for r in rows))),
        "traj_5y": (rows[min(5, len(rows) - 1)]["mu"] / max(1e-9, rows[0]["mu"])) - 1.0,
        "scarcity_leverage": rows[0]["R"],
    }
```

```
# ---------------------------------------------------------------------------
# 11. TRADE / PACKAGE MATH
# ---------------------------------------------------------------------------

def package_value(values: List[float], theta: float = 1.20,
                  roster_spot_cost: float = 120.0, spots_available: int = 3) -> float:
    """
    CES aggregation: (sum v^theta)^(1/theta), theta>1 creates the stud premium and
    prevents 3-for-1 quantity dumps from automatically 'winning'. Then charge for
    roster spots consumed beyond what the receiving team has free.
    """
    if not values:
        return 0.0
    core = sum(v ** theta for v in values) ** (1.0 / theta)
    overflow = max(0, len(values) - max(1, spots_available))
    return core - roster_spot_cost * overflow


def trade_verdict(side_a: List[float], side_b: List[float], **kw) -> Dict[str, float]:
    a, b = package_value(side_a, **kw), package_value(side_b, **kw)
    total = max(1.0, a + b)
    return {"side_a": a, "side_b": b, "edge": a - b, "edge_pct": 100.0 * (a - b) / total}


# ---------------------------------------------------------------------------
# 12. ROOKIE PICKS
# ---------------------------------------------------------------------------

# Illustrative outcome distribution by rookie-draft slot: (P(hit), value_if_hit,
# P(mid), value_if_mid). Values are in the same 0-10000 trade-value units.
# REPLACE with backtested class-specific numbers.
PICK_OUTCOMES = {
    1: (0.62, 7200, 0.20, 2600), 2: (0.55, 6400, 0.22, 2400),
    3: (0.50, 5900, 0.23, 2300), 4: (0.46, 5500, 0.24, 2200),
    6: (0.38, 5000, 0.26, 2000), 8: (0.33, 4600, 0.27, 1800),
    12: (0.27, 4200, 0.28, 1600), 16: (0.20, 3600, 0.28, 1300),
    24: (0.13, 3000, 0.26, 1000), 36: (0.07, 2400, 0.20, 700),
}


def pick_value(overall_slot: int, class_strength: float = 1.0,
               years_out: int = 0, strategy: str = "balanced") -> Dict[str, float]:
    keys = sorted(PICK_OUTCOMES)
    k = min(keys, key=lambda x: abs(x - overall_slot))
    p_hit, v_hit, p_mid, v_mid = PICK_OUTCOMES[k]
    ev = (p_hit * v_hit + p_mid * v_mid) * class_strength
    disc = STRATEGIES[strategy].discount ** years_out
    # contenders discount picks harder; rebuilders pay a flexibility premium
    flex_prem = {"contender": 0.92, "balanced": 1.00, "rebuilder": 1.08}[strategy]
    return {"ev": ev * disc * flex_prem,
            "p_hit": p_hit, "ceiling": v_hit * class_strength,
            "median": v_mid * class_strength, "p_miss": 1 - p_hit - p_mid}


# ---------------------------------------------------------------------------
# 13. ROSTER-LEVEL ANALYSIS
# ---------------------------------------------------------------------------

def roster_report(vals: List[Valuation], cfg: LeagueConfig) -> Dict[str, object]:
    by_group: Dict[str, List[Valuation]] = {}
    for v in vals:
        by_group.setdefault(cfg.group(v.pos), []).append(v)
    for g in by_group:
        by_group[g].sort(key=lambda v: v.raw["fpg"], reverse=True)

    starters_pts, surplus = 0.0, {}
    for g, need in cfg.starters.items():
        grp = by_group.get(g, [])
        starters_pts += sum(v.raw["fpg"] for v in grp[:need])
        have = sum(1 for v in grp if v.raw["vorg"] > 0)
        surplus[g] = have - need
```

```
    now = sum(v.trade_value["contender"] for v in vals)
    future = sum(v.trade_value["rebuilder"] for v in vals)
    ages = [v.raw["age"] for v in vals] or [0]
    window = "contend" if now > 1.15 * future else ("rebuild" if future > 1.15 * now else "retool")
    return {"projected_starter_ppg": starters_pts,
            "contender_capital": now, "rebuilder_capital": future,
            "now_vs_future_ratio": now / max(1.0, future),
            "avg_age": sum(ages) / len(ages),
            "positional_surplus": surplus, "recommended_direction": window}
```

Appendix B — `run_examples.py`

Driver that builds the synthetic replacement pool, defines the 13 hypothetical archetypes, calibrates the value scale, and prints everything in Part 10. Run it with `python3 run_examples.py` after saving Appendix A alongside it.

```
"""Worked examples for the dynasty valuation framework.

NOTE: the player pool used to derive replacement levels is SYNTHETIC (a smooth
rank-decay curve per position, calibrated to plausible 12-team Superflex/TEP/PPR
+ balanced-IDP scoring). Swap in the real projection file and every number below
recomputes. The 13 players are hypothetical archetypes, not real people.
"""
from dynasty_engine import (
    LeagueConfig, ReplacementEngine, synthetic_pool, DynastyModel, Player,
    RiskProfile, package_value, trade_verdict, pick_value, roster_report,
    STRATEGIES,
)

cfg = LeagueConfig()

POOL = {
    "QB": synthetic_pool(24.0, 0.020),
    "RB": synthetic_pool(20.0, 0.028),
    "WR": synthetic_pool(19.0, 0.024),
    "TE": synthetic_pool(16.0, 0.045),
    "DL": synthetic_pool(16.0, 0.030),
    "LB": synthetic_pool(19.0, 0.018),
    "DB": synthetic_pool(14.0, 0.020),
}

repl = ReplacementEngine(cfg, POOL)
print("=" * 78)
print("REPLACEMENT LEVELS  (12-tm SF / TEP / PPR / IDP  — synthetic pool)")
print("=" * 78)
for g in ["QB", "RB", "WR", "TE", "DL", "LB", "DB"]:
    print(f"  {g:>3}  startable slots league-wide: {repl.counts.get(g,0):>3}"
          f"   replacement FPG: {repl.R(g):6.2f}")

players = [
    Player("p1", "Young elite QB (dual)", "QB", 24.0, 3, 22.5, 16.5, 0.9, "dual",
           career_load=1150, kappa=0.06,
           risk=RiskProfile(0.92, 0.95, 0.85, 0.70, 1.8, 0.8, 0.08, 0.28, 0.0),
           market_value=9400, market_dispersion=0.12),
    Player("p2", "Aging productive QB (pocket)", "QB", 34.0, 12, 19.0, 16.0, 0.7, "pocket",
           career_load=6200, kappa=0.0,
           risk=RiskProfile(0.88, 0.80, 0.55, 0.62, 1.0, 0.7, 0.10, 0.25, 0.0),
           market_value=3600, market_dispersion=0.30),
    Player("p3", "Young RB, three-down", "RB", 23.0, 2, 15.5, 15.5, 1.1, None,
           career_load=310, kappa=0.18,
           risk=RiskProfile(0.80, 0.85, 0.80, 0.65, 1.1, 0.7, 0.15, 0.35, 0.0),
           market_value=7400, market_dispersion=0.22),
    Player("p4", "Aging RB, heavy mileage", "RB", 29.0, 8, 14.5, 15.0, 1.3, None,
           career_load=1720, kappa=0.0,
           risk=RiskProfile(0.72, 0.70, 0.35, 0.45, 0.9, 0.6, 0.20, 0.35, 0.0),
           market_value=2200, market_dispersion=0.34),
    Player("p5", "Young ascending WR", "WR", 23.0, 2, 14.0, 16.0, 1.4, None,
           career_load=140, kappa=0.30,
           risk=RiskProfile(0.72, 0.88, 0.85, 0.70, 0.7, 0.7, 0.22, 0.30, 0.0),
           market_value=7900, market_dispersion=0.26),
    Player("p6", "Veteran WR, stable role", "WR", 30.0, 9, 15.5, 16.0, 0.8, None,
           career_load=980, kappa=0.0,
           risk=RiskProfile(0.86, 0.70, 0.60, 0.68, 1.2, 0.7, 0.10, 0.28, 0.0),
```

```
            market_value=4100, market_dispersion=0.28),
    Player("p7", "Young TE, year-2 leap", "TE", 24.0, 2, 10.5, 16.0, 1.2, None,
            career_load=95, kappa=0.28,
            risk=RiskProfile(0.70, 0.80, 0.85, 0.70, 0.6, 0.7, 0.25, 0.35, 0.0),
            market_value=5600, market_dispersion=0.30),
    Player("p8", "Elite EDGE rusher", "EDGE", 26.0, 5, 14.5, 16.0, 0.9, None,
            career_load=2600, kappa=0.04,
            risk=RiskProfile(0.90, 0.90, 0.80, 0.68, 1.6, 0.8, 0.08, 0.55, 0.0),
            market_value=6200, market_dispersion=0.35),
    Player("p9", "Three-down LB, green dot", "LB", 25.0, 3, 17.5, 16.5, 0.8, None,
            career_load=2100, kappa=0.08,
            risk=RiskProfile(0.92, 0.75, 0.75, 0.72, 1.4, 0.85, 0.07, 0.20, 0.05),
            market_value=5400, market_dispersion=0.32),
    Player("p10", "Volatile boundary CB", "CB", 26.0, 4, 9.0, 16.0, 1.1, "boundary",
            career_load=2500, kappa=0.0,
            risk=RiskProfile(0.70, 0.65, 0.55, 0.62, 0.3, 0.6, 0.35, 0.70, 0.0),
            market_value=1800, market_dispersion=0.45),
    Player("p11", "Box safety, tackle floor", "S", 27.0, 5, 11.5, 16.0, 0.7, "box",
            career_load=2900, kappa=0.0,
            risk=RiskProfile(0.85, 0.55, 0.60, 0.70, 0.9, 0.65, 0.12, 0.25, 0.35),
            market_value=2600, market_dispersion=0.38),
    Player("p12", "Rookie WR, 1st-rd capital", "WR", 22.0, 1, 9.0, 15.0, 2.2, None,
            career_load=0, kappa=0.55,
            risk=RiskProfile(0.55, 0.95, 0.90, 0.60, -0.4, 0.6, 0.55, 0.35, 1.0),
            market_value=6800, market_dispersion=0.40),
    Player("p13", "Late-round breakout WR", "WR", 25.0, 3, 13.0, 16.0, 1.6, None,
            career_load=330, kappa=0.10,
            risk=RiskProfile(0.62, 0.15, 0.45, 0.66, 0.6, 0.6, 0.35, 0.35, 0.0),
            market_value=3900, market_dispersion=0.42),
]

model = DynastyModel(cfg, repl, stud_gamma=1.20, pool=POOL)
model.calibrate(players, target=9800.0)
vals = [model.value(p) for p in players]

print()
print("=" * 118)
print("WORKED EXAMPLES — fundamental trade values by strategy profile (0–10,000 scale)")
print("=" * 118)
hdr = (f"{'Player':<30}{'Pos':>4}{'Age':>5}{'FPG':>6}{'VORG':>7}"
       f"{'CONT':>7}{'BAL':>7}{'REBLD':>7}{'MKT':>7}{'GAP':>7}{'BLEND':>7}")
print(hdr)
print("-" * 118)
for v in vals:
    print(f"{v.name:<30}{v.pos:>4}{v.raw['age']:>5.0f}{v.raw['fpg']:>6.1f}"
          f"{v.raw['vorg']:>7.2f}"
          f"{v.trade_value['contender']:>7.0f}{v.trade_value['balanced']:>7.0f}"
          f"{v.trade_value['rebuilder']:>7.0f}"
          , end="")
    mv = next(p.market_value for p in players if p.player_id == v.player_id)
    print(f"{mv:>7.0f}{v.market_gap:>7.0f}{v.blended_value:>7.0f}")

print()
print("=" * 118)
print("PROBABILITY OUTPUTS")
print("=" * 118)
print(f"{'Player':<30}{'P(>repl)':>10}{'P(elite)':>10}{'P(start 3y)':>13}"
      f"{'P(breakout)':>13}{'P(collapse 1y)':>16}")
print("-" * 118)
for v in vals:
    p = v.probs
    print(f"{v.name:<30}{p['p_above_replacement']:>10.2f}{p['p_elite_season']:>10.2f}"
          f"{p['p_starter_in_3y']:>13.2f}{p['p_breakout']:>13.2f}"
          f"{p['p_value_collapse_1y']:>16.2f}")

print()
print("=" * 118)
print("VALUE DECOMPOSITION (balanced profile)")
print("=" * 118)
print(f"{'Player':<30}{'yr0':>8}{'yr1-2':>8}{'yr3+':>8}{'surv drag':>12}"
      f"{'5y traj':>12}{'age_eff':>9}")
print("-" * 118)
for v in vals:
```

```
    e = v.explain
    print(f"{v.name:<30}{e['year_0_share']:>8.0%}{e['years_1_2_share']:>8.0%}"
          f"{e['years_3_plus_share']:>8.0%}{e['survival_drag']:>12.0%}"
          f"{e['traj_5y']:>+12.0%}{v.raw['age_eff']:>9.1f}")

print()
print("=" * 78)
print("SEASON PATH — Young RB vs Aging RB (balanced horizon)")
print("=" * 78)
for v in [vals[2], vals[3]]:
    print(f"\n{v.name}:")
    print(f"  {'t':>2}{'age':>6}{'age_eff':>9}{'muFPG':>8}{'sigma':>7}"
          f"{'SSV':>9}{'S(t)':>7}{'E[val]':>9}")
    for r in v.seasons[:7]:
        print(f"  {int(r['t']):>2}{r['age']:>6.0f}{r['age_eff']:>9.1f}{r['mu']:>8.2f}"
              f"{r['sigma']:>7.2f}{r['ssv']:>9.1f}{r['survival']:>7.2f}{r['expected']:>9.1f}")

print()
print("=" * 78)
print("SCARCITY DEMO — LB vs S (the 'more points ≠ more value' requirement)")
print("=" * 78)
print(f"  LB replacement {repl.R('LB'):.2f} FPG | DB replacement {repl.R('DB'):.2f} FPG")
print(f"  Three-down LB @17.5 FPG -> VORG {17.5-repl.R('LB'):.2f}"
      f"  | trade value {vals[8].trade_value['balanced']:.0f}")
print(f"  Box safety   @11.5 FPG -> VORG {11.5-repl.R('DB'):.2f}"
      f"  | trade value {vals[10].trade_value['balanced']:.0f}")
mid_lb = 12.5
print(f"  Mid LB       @{mid_lb} FPG -> VORG {mid_lb-repl.R('LB'):.2f}"
      f"  (scores MORE than the safety, is worth LESS)")

print()
print("=" * 78)
print("TRADE MATH — 2-for-1 consolidation (theta=1.20)")
print("=" * 78)
young_wr = vals[4].trade_value["balanced"]
vet_wr = vals[5].trade_value["balanced"]
young_te = vals[6].trade_value["balanced"]
print(f"  Side A: young ascending WR ({young_wr:.0f})")
print(f"  Side B: veteran WR ({vet_wr:.0f}) + young TE ({young_te:.0f}) "
      f"= naive sum {vet_wr+young_te:.0f}")
tv = trade_verdict([young_wr], [vet_wr, young_te], spots_available=2)
print(f"  CES package value  A={tv['side_a']:.0f}  B={tv['side_b']:.0f}"
      f"  edge to A: {tv['edge']:+.0f} ({tv['edge_pct']:+.1f}%)")
tv2 = trade_verdict([young_wr], [vet_wr, young_te], spots_available=1)
print(f"  Same trade, receiving team has 1 open roster spot: "
       f"B={tv2['side_b']:.0f} (roster-spot charge applied)")

print()
print("=" * 78)
print("ROOKIE PICKS")
print("=" * 78)
for slot in [1, 3, 6, 12, 16, 24, 36]:
    for strat in ["contender", "balanced", "rebuilder"]:
        pv = pick_value(slot, class_strength=1.0, years_out=0, strategy=strat)
        if strat == "balanced":
            bal = pv
        if strat == "contender":
            con = pv
        if strat == "rebuilder":
            reb = pv
    print(f"  pick {slot:>2}: contender {con['ev']:>6.0f} | balanced {bal['ev']:>6.0f}"
          f" | rebuilder {reb['ev']:>6.0f} | P(hit) {bal['p_hit']:.2f}"
          f" | ceiling {bal['ceiling']:.0f}")
nxt = pick_value(6, class_strength=0.9, years_out=1, strategy="rebuilder")
print(f"  next-year pick ~1.06, weak class (0.9x), rebuilder: {nxt['ev']:.0f}")

print()
print("=" * 78)
print("ROSTER REPORT — sample 13-player roster")
print("=" * 78)
rep = roster_report(vals, cfg)
for k, v in rep.items():
    print(f"  {k}: {v}")
```

Appendix C — `examples_output.txt`

Verbatim output of Appendix B. This is the source of every number in Part 10 — nothing there was computed by hand.

```
================================================================================
REPLACEMENT LEVELS  (12-tm SF / TEP / PPR / IDP  — synthetic pool)
================================================================================
   QB  startable slots league-wide:  24   replacement FPG:  12.40
   RB  startable slots league-wide:  33   replacement FPG:   6.35
   WR  startable slots league-wide:  36   replacement FPG:   6.15
   TE  startable slots league-wide:  15   replacement FPG:   6.80
   DL  startable slots league-wide:  24   replacement FPG:   6.70
   LB  startable slots league-wide:  48   replacement FPG:   7.32
   DB  startable slots league-wide:  36   replacement FPG:   6.17

================================================================================================================================
WORKED EXAMPLES — fundamental trade values by strategy profile (0-10,000 scale)
================================================================================================================================
Player                          Pos  Age   FPG   VORG   CONT    BAL  REBLD    MKT    GAP  BLEND
--------------------------------------------------------------------------------------------------------------------------------
Young elite QB (dual)            QB   24  22.5  10.10   9800   7974   6265   9400  -1426   8723
Aging productive QB (pocket)     QB   34  19.0   6.60   3458   2112   1128   3600  -1488   2856
Young RB, three-down             RB   23  15.5   9.15   8492   6615   4982   7400   -785   7017
Aging RB, heavy mileage          RB   29  14.5   8.15   2674   1406    520   2200   -794   1798
Young ascending WR               WR   23  14.0   7.85   9693   9800   9800   7900   1900   8839
Veteran WR, stable role          WR   30  15.5   9.35   4368   2485   1144   4100  -1615   3297
Young TE, year-2 leap            TE   24  10.5   3.70   5875   6367   6782   5600    767   5983
Elite EDGE rusher              EDGE   26  14.5   7.80   6301   4712   3387   6200  -1488   5164
Three-down LB, green dot         LB   25  17.5  10.18   8946   6661   4750   5400   1261   6267
Volatile boundary CB             CB   26   9.0   2.83   2055   1328    786   1800   -472   1465
Box safety, tackle floor          S   27  11.5   5.33   3239   2087   1228   2600   -513   2258
Rookie WR, 1st-rd capital        WR   22   9.0   2.85   6160   7051   7958   6800    251   6935
Late-round breakout WR           WR   25  13.0   6.85   6140   4857   3719   3900    957   4396

================================================================================================================================
PROBABILITY OUTPUTS
================================================================================================================================
Player                          P(>repl)  P(elite)  P(start 3y)  P(breakout)  P(collapse 1y)
--------------------------------------------------------------------------------------------------------------------------------
Young elite QB (dual)               0.92      0.54         0.75         0.22            0.13
Aging productive QB (pocket)        0.86      0.33         0.24         0.08            0.35
Young RB, three-down                0.89      0.40         0.58         0.36            0.21
Aging RB, heavy mileage             0.87      0.35         0.01         0.01            0.65
Young ascending WR                  0.88      0.33         0.73         0.45            0.16
Veteran WR, stable role             0.92      0.42         0.20         0.05            0.35
Young TE, year-2 leap               0.75      0.34         0.66         0.51            0.24
Elite EDGE rusher                   0.88      0.54         0.61         0.27            0.21
Three-down LB, green dot            0.97      0.51         0.66         0.24            0.12
Volatile boundary CB                0.67      0.28         0.23         0.30            0.48
Box safety, tackle floor            0.85      0.41         0.35         0.21            0.30
Rookie WR, 1st-rd capital           0.63      0.18         0.55         0.52            0.38
Late-round breakout WR              0.84      0.29         0.57         0.34            0.23

================================================================================================================================
VALUE DECOMPOSITION (balanced profile)
================================================================================================================================
Player                            yr0   yr1-2    yr3+   surv drag     5y traj  age_eff
--------------------------------------------------------------------------------------------------------------------------------
Young elite QB (dual)             27%     44%     29%         10%        -26%     24.0
Aging productive QB (pocket)      53%     41%      7%         25%        -53%     35.0
Young RB, three-down              27%     48%     25%         24%        -37%     23.0
Aging RB, heavy mileage           85%     15%      0%         15%        -95%     30.9
Young ascending WR                17%     40%     43%         21%        +52%     23.0
Veteran WR, stable role           63%     34%      2%         17%        -83%     31.1
Young TE, year-2 leap             14%     40%     46%         24%        +73%     24.0
Elite EDGE rusher                 32%     46%     22%         18%        -43%     26.0
Three-down LB, green dot          31%     47%     22%         18%        -43%     25.0
Volatile boundary CB              46%     47%      8%         26%        -69%     26.0
Box safety, tackle floor          44%     47%      9%         22%        -66%     27.0
Rookie WR, 1st-rd capital         13%     38%     49%         29%       +117%     22.0
Late-round breakout WR            28%     46%     26%         27%        -20%     25.0

================================================================================
```

```
SEASON PATH — Young RB vs Aging RB (balanced horizon)
================================================================================

Young RB, three-down:
   t   age  age_eff   muFPG  sigma      SSV   S(t)   E[val]
   0    23     23.0   15.50   7.41    147.9   1.00    147.9
   1    24     24.0   17.48   9.85    185.0   0.91    167.8
   2    25     25.2   18.40  11.74    204.2   0.82    167.9
   3    26     26.7   17.27  12.18    191.1   0.71    135.6
   4    27     28.2   13.96  10.73    143.5   0.58     82.8
   5    28     29.7    9.78   8.10     82.4   0.44     36.3
   6    29     31.2    5.95   5.31     30.3   0.30      9.2

Aging RB, heavy mileage:
   t   age  age_eff   muFPG  sigma      SSV   S(t)   E[val]
   0    29     30.9   14.50   7.25    129.4   1.00    129.4
   1    30     32.5    7.87   4.72     42.2   0.56     23.8
   2    31     33.5    4.78   3.37     10.8   0.32      3.4
   3    32     34.5    2.79   2.38      1.1   0.18      0.2
   4    33     35.5    1.53   1.76      0.0   0.10      0.0
   5    34     36.5    0.79   1.46      0.0   0.06      0.0
   6    35     37.5    0.38   1.34      0.0   0.03      0.0

================================================================================
SCARCITY DEMO — LB vs S (the 'more points ≠ more value' requirement)
================================================================================
  LB replacement 7.32 FPG | DB replacement 6.17 FPG
  Three-down LB @17.5 FPG -> VORG 10.18  | trade value 6661
  Box safety   @11.5 FPG -> VORG 5.33  | trade value 2087
  Mid LB       @12.5 FPG -> VORG 5.18  (scores MORE than the safety, is worth LESS)

================================================================================
TRADE MATH — 2-for-1 consolidation (theta=1.20)
================================================================================
  Side A: young ascending WR (9800)
  Side B: veteran WR (2485) + young TE (6367) = naive sum 8852
  CES package value  A=9800  B=8041  edge to A: +1759 (+9.9%)
  Same trade, receiving team has 1 open roster spot: B=7921 (roster-spot charge applied)

================================================================================
ROOKIE PICKS
================================================================================
  pick  1: contender   4585 | balanced   4984 | rebuilder   5383 | P(hit) 0.62 | ceiling 7200
  pick  3: contender   3201 | balanced   3479 | rebuilder   3757 | P(hit) 0.50 | ceiling 5900
  pick  6: contender   2226 | balanced   2420 | rebuilder   2614 | P(hit) 0.38 | ceiling 5000
  pick 12: contender   1455 | balanced   1582 | rebuilder   1709 | P(hit) 0.27 | ceiling 4200
  pick 16: contender    997 | balanced   1084 | rebuilder   1171 | P(hit) 0.20 | ceiling 3600
  pick 24: contender    598 | balanced    650 | rebuilder    702 | P(hit) 0.13 | ceiling 3000
  pick 36: contender    283 | balanced    308 | rebuilder    333 | P(hit) 0.07 | ceiling 2400
  next-year pick ~1.06, weak class (0.9x), rebuilder: 2188

================================================================================
ROSTER REPORT — sample 13-player roster
================================================================================
  projected_starter_ppg: 158.0
  contender_capital: 77200.44208318304
  rebuilder_capital: 52449.00468335995
  now_vs_future_ratio: 1.471914339447432
  avg_age: 26.0
  positional_surplus: {'QB': 1, 'RB': 0, 'WR': 1, 'TE': 0, 'DL': -1, 'LB': -2, 'DB': -1}
  recommended_direction: contend
```

Appendix D — `schema.sql`

PostgreSQL DDL. The design rule throughout: every fact that can change carries `as_of`, every derived value carries `(model_version, param_set_id, config_id, as_of)`, and nothing is ever UPDATEd in place.

```
-- ================================================================================
-- Brisket Dynasty Valuation Model — PostgreSQL schema v1.0
-- Design rule: every fact that can change over time carries as_of.
--              every derived value carries (model_version, param_set_id, config_id, as_of).
--              nothing is ever UPDATEd in place — value history is the product.
```

```
-- ============================================================================

CREATE SCHEMA IF NOT EXISTS bdvm;
SET search_path = bdvm, public;

-- ------------------------------------------------------------------ identity --

CREATE TABLE players (
    player_id        TEXT PRIMARY KEY,              -- GSIS id preferred
    full_name        TEXT NOT NULL,
    birth_date       DATE,
    pos_true         TEXT NOT NULL CHECK (pos_true IN
                       ('QB','RB','WR','TE','DT','EDGE','LB','CB','S')),
    archetype        TEXT,                          -- pocket/dual, box/deep, slot/boundary
    archetype_weight NUMERIC,                       -- continuous blend, 0-1
    entry_year       INT,
    draft_round      INT,
    draft_pick       INT,                           -- overall
    college          TEXT,
    height_in        INT,
    weight_lb        INT,
    created_at       TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE player_ids (
    player_id  TEXT REFERENCES players(player_id),
    source     TEXT NOT NULL,                       -- sleeper, ktc, fantasycalc, pfr, ...
    source_id  TEXT NOT NULL,
    PRIMARY KEY (player_id, source)
);

-- Platform position can differ from true position and can change between seasons.
CREATE TABLE player_designations (
    player_id       TEXT REFERENCES players(player_id),
    season          INT NOT NULL,
    platform        TEXT NOT NULL,                  -- sleeper, mfl, ffpc...
    pos_platform    TEXT NOT NULL,
    as_of           DATE NOT NULL,
    PRIMARY KEY (player_id, season, platform, as_of)
);

-- ------------------------------------------------------------- nfl history --

CREATE TABLE player_seasons (
    player_id       TEXT REFERENCES players(player_id),
    season          INT NOT NULL,
    team            TEXT,
    games           INT,
    games_started   INT,
    snap_pct        NUMERIC,
    route_pct       NUMERIC,
    tgt_share       NUMERIC,
    carry_share     NUMERIC,
    goalline_share  NUMERIC,
    career_load     NUMERIC,                        -- touches/targets/dropbacks/snaps to date
    -- offense raw
    pass_att INT, pass_yd INT, pass_td INT, interceptions INT,
    rush_att INT, rush_yd INT, rush_td INT,
    targets INT, receptions INT, rec_yd INT, rec_td INT, first_downs INT,
    -- idp raw
    tkl_solo INT, tkl_ast INT, sacks NUMERIC, tfl INT, qb_hits INT,
    pass_def INT, int_def INT, forced_fum INT, fum_rec INT, def_td INT,
    pass_rush_snaps INT, box_snaps INT, slot_snaps INT, deep_snaps INT,
    coverage_snaps INT, targets_faced INT,
    -- derived / charted
    yprr NUMERIC, tprr NUMERIC, pressure_rate NUMERIC,
    expected_sacks NUMERIC, tackle_rate NUMERIC, tackles_vs_expected NUMERIC,
    exp_fpts NUMERIC,                               -- opportunity-based expected points
    as_of           DATE NOT NULL,
    PRIMARY KEY (player_id, season, as_of)
);

CREATE TABLE player_injuries (
```

```
    player_id   TEXT REFERENCES players(player_id),
    season      INT, week INT,
    body_part   TEXT, injury_type TEXT,             -- soft_tissue / joint / bone / concussion
    games_missed INT,
    as_of       DATE NOT NULL,
    PRIMARY KEY (player_id, season, week, as_of)
);

CREATE TABLE player_contracts (
    player_id            TEXT REFERENCES players(player_id),
    season               INT,
    years_left           INT,
    guaranteed_remaining NUMERIC,
    dead_cap_if_cut      NUMERIC,
    fifth_year_option    BOOLEAN,
    franchise_tag        BOOLEAN,
    status               TEXT,                      -- active/ir/ps/fa/rfa
    as_of                DATE NOT NULL,
    PRIMARY KEY (player_id, season, as_of)
);

CREATE TABLE college_production (
    player_id        TEXT PRIMARY KEY REFERENCES players(player_id),
    breakout_age     NUMERIC,
    peak_dominator   NUMERIC,
    career_dominator NUMERIC,
    yds_per_team_att NUMERIC,
    early_declare    BOOLEAN,
    conference       TEXT,
    ras              NUMERIC,                       -- athletic score
    col_pressure_rate NUMERIC, col_tackle_rate NUMERIC, col_coverage_grade NUMERIC
);

-- ------------------------------------------------------------ projections --

CREATE TABLE projection_sources (
    source        TEXT PRIMARY KEY,
    display_name  TEXT,
    is_consensus  BOOLEAN DEFAULT false,
    active        BOOLEAN DEFAULT true
);

CREATE TABLE source_weights (                       -- trained ONLY on prior seasons
    source     TEXT REFERENCES projection_sources(source),
    pos        TEXT NOT NULL,
    season     INT  NOT NULL,                       -- season the weight applies TO
    mase       NUMERIC,
    weight     NUMERIC NOT NULL,
    trained_through_season INT NOT NULL,            -- leakage guard
    PRIMARY KEY (source, pos, season)
);

CREATE TABLE projections_raw (
    id          BIGSERIAL PRIMARY KEY,
    player_id   TEXT REFERENCES players(player_id),
    source      TEXT REFERENCES projection_sources(source),
    season      INT NOT NULL,
    stat_line   JSONB,                              -- raw projected stats
    fpts        NUMERIC,                            -- fallback if no stat_line
    games       NUMERIC,
    proj_high   NUMERIC, proj_low NUMERIC,
    scoring_native BOOLEAN DEFAULT false,           -- true if fpts already in our scoring
    as_of       DATE NOT NULL,
    UNIQUE (player_id, source, season, as_of)
);

CREATE TABLE projection_blend (
    player_id    TEXT REFERENCES players(player_id),
    season       INT NOT NULL,
    config_id    TEXT NOT NULL,
    fpg          NUMERIC NOT NULL,
    sigma_source NUMERIC NOT NULL,
    games        NUMERIC NOT NULL,
```

```
    n_sources    INT,
    imputed      BOOLEAN DEFAULT false,
    as_of        DATE NOT NULL,
    PRIMARY KEY (player_id, season, config_id, as_of)
);

-- ------------------------------------------------------------ leagues ---

CREATE TABLE league_configs (
    config_id     TEXT PRIMARY KEY,
    display_name  TEXT,
    teams         INT NOT NULL,
    scoring       JSONB NOT NULL,
    starters      JSONB NOT NULL,
    flex_slots    JSONB NOT NULL,
    waiver_buffer JSONB NOT NULL,
    pos_groups    JSONB NOT NULL,                  -- true pos -> lineup group
    roster_size   INT, taxi_size INT,
    best_ball     BOOLEAN DEFAULT false,
    config_hash   TEXT NOT NULL,
    created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE replacement_levels (
    config_id   TEXT REFERENCES league_configs(config_id),
    season      INT NOT NULL,
    pos_group   TEXT NOT NULL,
    slots       INT NOT NULL,                      -- startable league-wide
    repl_rank   INT NOT NULL,
    repl_fpg    NUMERIC NOT NULL,
    as_of       DATE NOT NULL,
    PRIMARY KEY (config_id, season, pos_group, as_of)
);

-- ------------------------------------------------------------ model ----

CREATE TABLE model_params (
    param_set_id  SERIAL PRIMARY KEY,
    label         TEXT,
    age_curves    JSONB NOT NULL,                  -- pos -> [peak, c_up, c_dn]
    mileage       JSONB NOT NULL,
    hazards       JSONB NOT NULL,
    cv_base       JSONB NOT NULL,
    drift_vol     JSONB NOT NULL,
    strategies    JSONB NOT NULL,                  -- d, horizon, usability, lambda
    misc          JSONB NOT NULL,                  -- gamma, theta, decays, regressions
    backtest_ref  TEXT,                            -- git tag / experiment id justifying it
    created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE player_values (
    id            BIGSERIAL PRIMARY KEY,
    player_id     TEXT REFERENCES players(player_id),
    season        INT NOT NULL,
    config_id     TEXT REFERENCES league_configs(config_id),
    strategy      TEXT NOT NULL CHECK (strategy IN ('contender','balanced','rebuilder')),
    dv_raw        NUMERIC NOT NULL,                -- pre-scale discounted value
    trade_value   NUMERIC NOT NULL,                -- 0-10000
    fpg           NUMERIC, repl_fpg NUMERIC, vorg NUMERIC,
    age_eff       NUMERIC,
    p_above_repl  NUMERIC, p_elite NUMERIC, p_starter_3y NUMERIC,
    p_breakout    NUMERIC, p_collapse_1y NUMERIC,
    floor_p20     NUMERIC, ceiling_p85 NUMERIC,
    confidence    NUMERIC, volatility NUMERIC,
    explain       JSONB,
    data_quality  NUMERIC,                         -- 0-1, drops with imputation
    model_version TEXT NOT NULL,
    param_set_id  INT REFERENCES model_params(param_set_id),
    as_of         DATE NOT NULL,
    UNIQUE (player_id, season, config_id, strategy, model_version, param_set_id, as_of)
);
CREATE INDEX ON player_values (config_id, strategy, as_of, trade_value DESC);
CREATE INDEX ON player_values (player_id, as_of);
```

```
CREATE TABLE value_paths (                          -- career-value chart data
    player_id     TEXT, season INT, config_id TEXT, strategy TEXT,
    t             INT NOT NULL,
    age           NUMERIC, age_eff NUMERIC,
    mu_fpg        NUMERIC, sigma NUMERIC, games NUMERIC,
    ssv           NUMERIC, survival NUMERIC, expected NUMERIC,
    model_version TEXT, param_set_id INT, as_of DATE NOT NULL,
    PRIMARY KEY (player_id, season, config_id, strategy, t, as_of)
);

-- ------------------------------------------------------------ market ----

CREATE TABLE market_sources (
    source     TEXT PRIMARY KEY,
    mkt_type   TEXT NOT NULL CHECK (mkt_type IN ('crowd','trade_derived','expert')),
    weight     NUMERIC DEFAULT 1.0
);

CREATE TABLE market_values (
    player_id  TEXT REFERENCES players(player_id),
    source     TEXT REFERENCES market_sources(source),
    format     TEXT NOT NULL,                       -- 1qb / sf, tep tier, idp flag
    value      NUMERIC NOT NULL,                    -- normalised 0-10000
    raw_value  NUMERIC,
    as_of      DATE NOT NULL,
    PRIMARY KEY (player_id, source, format, as_of)
);

CREATE TABLE market_gaps (
    player_id     TEXT, config_id TEXT, season INT,
    model_value   NUMERIC, market_consensus NUMERIC,
    dispersion    NUMERIC, liquidity NUMERIC,
    gap           NUMERIC, alpha NUMERIC,
    momentum_30d  NUMERIC,
    gap_first_seen DATE,                             -- persistence guard for BUY signals
    signal        TEXT,
    blended_value NUMERIC,
    as_of         DATE NOT NULL,
    PRIMARY KEY (player_id, config_id, season, as_of)
);

-- ------------------------------------------------------------ news events ---

CREATE TABLE news_event_types (
    event_type       TEXT PRIMARY KEY,
    default_impact   JSONB NOT NULL,                 -- mu_pct, role_delta, sigma_mult, hazard_mult
    default_half_life INT NOT NULL
);

CREATE TABLE news_events (
    event_id       TEXT PRIMARY KEY,
    player_id      TEXT REFERENCES players(player_id),
    event_type     TEXT REFERENCES news_event_types(event_type),
    effective_date DATE NOT NULL,
    confidence     NUMERIC NOT NULL CHECK (confidence BETWEEN 0 AND 1),
    source_reliability NUMERIC NOT NULL,
    already_in_projection BOOLEAN NOT NULL DEFAULT false,
    impact         JSONB,                            -- overrides default_impact
    duration_days  INT, half_life_days INT,
    notes          TEXT,
    created_at     TIMESTAMPTZ DEFAULT now()
);

-- ------------------------------------------------------------ rookie picks ---

CREATE TABLE rookie_pick_outcomes (
    format        TEXT NOT NULL,                     -- sf_tep_idp, 1qb_ppr, ...
    overall_slot  INT  NOT NULL,
    p_hit NUMERIC, v_hit NUMERIC,
    p_mid NUMERIC, v_mid NUMERIC,
    p_miss NUMERIC, v_miss NUMERIC,
    sample_n INT, seasons_covered TEXT,
```

```
    param_set_id INT REFERENCES model_params(param_set_id),
    PRIMARY KEY (format, overall_slot, param_set_id)
);

CREATE TABLE draft_class_strength (
    class_year INT, format TEXT, position TEXT,
    strength   NUMERIC,                          -- multiplier ~0.85-1.15
    as_of      DATE NOT NULL,
    PRIMARY KEY (class_year, format, position, as_of)
);

-- ------------------------------------------------------------ backtests ---

CREATE TABLE model_experiments (
    experiment_id SERIAL PRIMARY KEY,
    label         TEXT NOT NULL,
    param_set_id  INT REFERENCES model_params(param_set_id),
    ablation      TEXT,                          -- what was turned off
    target        TEXT NOT NULL,                 -- T1..T8
    fold          TEXT NOT NULL,
    pos           TEXT,
    metric        TEXT NOT NULL,                 -- spearman, mae, brier, ndcg@24
    value         NUMERIC NOT NULL,
    baseline      TEXT,                          -- B0..B4
    baseline_value NUMERIC,
    n_rows        INT,
    run_at        TIMESTAMPTZ DEFAULT now()
);

-- ------------------------------------------------------------ user layer ---

CREATE TABLE user_rosters (
    user_id    TEXT, league_id TEXT, config_id TEXT,
    player_id  TEXT REFERENCES players(player_id),
    acquired_at DATE,
    as_of      DATE NOT NULL,
    PRIMARY KEY (user_id, league_id, player_id, as_of)
);

CREATE TABLE roster_reports (
    user_id TEXT, league_id TEXT, config_id TEXT,
    starter_ppg NUMERIC,
    contender_capital NUMERIC, rebuilder_capital NUMERIC, now_future_ratio NUMERIC,
    window_peak_season INT,
    positional_surplus JSONB,
    age_concentration NUMERIC, risk_concentration NUMERIC, liquidity NUMERIC,
    direction TEXT,                              -- contend / retool / rebuild
    as_of DATE NOT NULL,
    PRIMARY KEY (user_id, league_id, as_of)
);
```

Appendix E — `league_config.example.json`

Your league encoded as config. The `pos_groups` and `waiver_buffer` objects are the two things to change when supporting a different format — everything else in the model is downstream of them.

```
{
  "config_id": "brisket_sf_tep_idp_12",
  "display_name": "12-team Superflex / TE Premium / PPR / IDP dynasty (Sleeper)",
  "teams": 12,
  "roster_size": 35,
  "taxi_size": 5,
  "best_ball": false,

  "starters": { "QB": 1, "RB": 2, "WR": 3, "TE": 1, "DL": 2, "LB": 3, "DB": 3 },
  "flex_slots": {
    "FLEX":      { "count": 1, "eligible": ["RB", "WR", "TE"] },
    "SUPERFLEX": { "count": 1, "eligible": ["QB", "RB", "WR", "TE"] },
    "IDP_FLEX":  { "count": 1, "eligible": ["DL", "LB", "DB"] }
  },

  "_comment_pos_groups": "Map TRUE position -> lineup group. Change DT/EDGE to separate groups for a DT-required league, or split CB/S
```

```
  "pos_groups": {
    "QB": "QB", "RB": "RB", "WR": "WR", "TE": "TE",
    "DT": "DL", "EDGE": "DL", "LB": "LB", "CB": "DB", "S": "DB"
  },

  "_comment_buffer": "Players per team rostered AND startable-quality beyond the startable count. This sets the origin of the value sc
  "waiver_buffer": {
    "QB": 0.85, "RB": 0.75, "WR": 1.00, "TE": 0.40,
    "DL": 0.50, "LB": 0.50, "DB": 0.50
  },

  "scoring": {
    "pass_yd": 0.04, "pass_td": 4.0, "interception": -2.0,
    "rush_yd": 0.1,  "rush_td": 6.0,
    "rec": 1.0, "rec_te_bonus": 0.5, "rec_yd": 0.1, "rec_td": 6.0,
    "fumble_lost": -2.0, "first_down": 0.0,

    "tackle_solo": 1.5, "tackle_assist": 0.75,
    "sack": 4.0, "tfl": 1.0, "qb_hit": 0.5,
    "pass_defended": 1.5, "interception_def": 4.0,
    "forced_fumble": 3.0, "fumble_recovery": 2.0,
    "def_td": 6.0, "safety": 2.0
  },

  "strategy_profiles": {
    "contender": { "discount": 0.72, "horizon": 4,
                   "usability": [1.00, 0.95, 0.80, 0.65, 0.55],
                   "risk_lambda": 0.20,  "pick_premium": 0.92 },
    "balanced":  { "discount": 0.85, "horizon": 6,
                   "usability": [1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80],
                   "risk_lambda": 0.10,  "pick_premium": 1.00 },
    "rebuilder": { "discount": 0.93, "horizon": 8,
                   "usability": [0.45, 0.75, 1.00, 1.00, 1.00, 0.95, 0.90, 0.85, 0.80],
                   "risk_lambda": -0.10, "pick_premium": 1.08 }
  },

  "scale": { "target_top_value": 9800, "stud_gamma": 1.20,
             "package_theta": 1.20, "roster_spot_cost": 120 }
}
```

Appendix F — `player_payload.example.json`

The API contract. `_request` shows what the projection ingest accepts (raw stat lines preferred, fantasy points as fallback); `_response` shows the full player-card payload including path, probabilities and explanation.

```
{
  "_request": {
    "league_config_id": "brisket_sf_tep_idp_12",
    "season": 2026,
    "as_of": "2026-07-27",
    "strategies": ["contender", "balanced", "rebuilder"],
    "include": ["path", "probs", "explain", "market"],
    "players": [
      {
        "player_id": "00-0000000",
        "pos_true": "WR",
        "archetype": null,
        "birth_date": "2003-02-14",
        "nfl_season": 2,
        "draft_round": 1, "draft_pick": 18,
        "career_load": 140,

        "_comment": "Provide EITHER stat_line (preferred) OR fpts per source.",
        "projections": [
          { "source": "SourceA", "games": 16.0,
            "stat_line": { "rec": 82, "rec_yd": 1120, "rec_td": 7,
                           "rush_yd": 40, "rush_td": 0, "fum_lost": 1 } },
          { "source": "SourceB", "games": 16.0, "fpts": 229.0, "scoring_native": false },
          { "source": "SourceC", "games": 15.5,
            "stat_line": { "rec": 76, "rec_yd": 1015, "rec_td": 6 },
            "proj_high": 268.0, "proj_low": 154.0 }
        ],
```

```
      "role": { "route_pct": 0.68, "tgt_share": 0.21, "tprr": 0.22,
                "snap_pct": 0.74, "vacated_tgt_share": 0.09 },

      "risk": { "role_security": 0.72, "draft_capital": 0.88,
                "contract_security": 0.85, "durability": 0.70,
                "production_z": 0.7, "scheme_stability": 0.7,
                "role_uncertainty": 0.22, "event_volatility": 0.30,
                "small_sample": 0.0, "designation_risk": 0.0 },

      "college": { "breakout_age": 19.6, "peak_dominator": 0.34, "early_declare": true },

      "market": { "consensus": 7900, "dispersion": 0.26, "momentum_30d": -0.03,
                  "sources": { "ktc": 8100, "fantasycalc": 7600, "expert_consensus": 8000 } }
    }
  ]
 },

 "_response": {
  "meta": { "model_version": "1.0.0", "param_set_id": 42,
            "config_id": "brisket_sf_tep_idp_12", "as_of": "2026-07-27" },
  "players": [
    {
      "player_id": "00-0000000", "pos": "WR", "age": 23.4,
      "projection": { "fpg": 14.0, "games": 16.0, "sigma_source": 1.4,
                      "n_sources": 3, "imputed": false },
      "replacement": { "group": "WR", "fpg": 6.15, "rank": 48 },
      "values": { "contender": 9693, "balanced": 9800, "rebuilder": 9800 },
      "value_raw": { "contender": 512.4, "balanced": 604.1, "rebuilder": 803.9 },
      "market": { "consensus": 7900, "dispersion": 0.26,
                  "gap": 1900, "liquidity": 0.77, "alpha": 1463,
                  "blended": 8839, "signal": "STRONG_BUY",
                  "gap_first_seen": "2026-07-05" },
      "probs": { "above_replacement": 0.88, "elite_season": 0.33,
                 "starter_in_3y": 0.73, "bust_year1": 0.12,
                 "breakout": 0.45, "collapse_1y": 0.16 },
      "range": { "floor_p20": 6900, "median": 9800, "ceiling_p85": 12400,
                 "confidence": 0.58, "volatility": 0.71 },
      "path": [
        { "t": 0, "age": 23.4, "age_eff": 23.4, "mu": 14.00, "sigma": 4.90,
          "games": 16.0, "ssv": 128.4, "survival": 1.00, "expected": 128.4 },
        { "t": 1, "age": 24.4, "age_eff": 24.4, "mu": 16.61, "sigma": 6.42,
          "games": 16.6, "ssv": 176.8, "survival": 0.88, "expected": 155.6 },
        { "t": 2, "age": 25.4, "age_eff": 25.6, "mu": 18.02, "sigma": 7.71,
          "games": 16.6, "ssv": 199.6, "survival": 0.79, "expected": 157.7 }
      ],
      "explain": {
        "year_0_share": 0.17, "years_1_2_share": 0.40, "years_3_plus_share": 0.43,
        "survival_drag": 0.21, "traj_5y": 0.52, "scarcity_leverage": 6.15,
        "top_drivers": [
          { "driver": "role_growth", "direction": "+", "magnitude": 0.31 },
          { "driver": "age_curve",   "direction": "+", "magnitude": 0.19 },
          { "driver": "positional_scarcity", "direction": "+", "magnitude": 0.12 },
          { "driver": "role_uncertainty", "direction": "-", "magnitude": 0.09 }
        ],
        "narrative": "Model has him WR4; market has him WR9. The gap is role growth the market hasn't paid for. His 68% route share
      },
      "flags": ["ASCENSION_UNCONFIRMED", "BUY_CANDIDATE"],
      "data_quality": 0.92
    }
  ]
 }
}
```